CMS · NanoAOD · trigger objects

The unseeded diphoton leg never gets its NanoAOD filter bits

Photon `TrigObj_filterBits` 16, 17 (14, 15 before NanoAOD v15) of `HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90` / `Mass95`: documented, never set. Cause, scope, the one-line fix and its test on real input.

Santanu Mahata

5 November 2026

Checked against cms-sw/cmssw master ed35654b1, release branches 13_0_X to 16_1_X, HLT menus 2018 to 2026, twelve NanoAOD files and a NANO re-run with the fix

Outline

1. **The problem** — what a NanoAOD trigger object is, the diphoton path, where the object is dropped.
2. **Scope** — every NanoAOD version since v12, every menu since 2018, twelve files measured, a scan of the whole trigger-object table.
3. **Fix and validation** — the one-line fix, the fix test on real MiniAOD input, the relval in CMSSW_16_1_3, impact on analyses, pull-request status.
4. **Backup** — the code at every step, all tables, history, production recipe, reproducibility.

Repository: `https://github.com/santanumahata/cmssw/tree/nanoaod-trigobj-notes` (NOTES.md, results/REPORT.md, scripts/).

Part I

The problem

The problem in one page

3

photon filter bits documented but impossible to set (16, 17, 19 in v15; 14, 15, 17 in v12 to v14)

4+

NanoAOD versions affected: v12, v13, v14, v15 and master

19

HLT menu files checked, 2018 to 2026: same filter names, same collections

1

line to change in `triggerObjects_cff.py`, as the Electron selection did in 2023

- **Problem**: the Photon entry of the trigger-object table accepts only objects from `hltEgammaCandidates`; the sub-leading leg of the Diphoton30_22 Mass90/Mass95 paths is built on `hltEgammaCandidatesUnseeded`. The collection test runs before the bits are computed, so those objects are dropped and their bits can never appear.
- **Measured**: twelve NanoAOD files, v9 to v15: not one photon object with an unseeded bit.
- **Fix**: accept both collections in the Photon selection. NANO re-made with the fix sets the unseeded bits in all 10453 Mass90 events of the test file and changes nothing else.

NOTES.md sections 1, 7, 14, 15; results/REPORT.md.

What a NanoAOD trigger object is, and how analyses use the filter bits

branch	meaning
`TrigObj_id`	object type: 11 electron, 22 photon, 13 muon, 15 tau, 1 jet, 6 fat jet, 2 MET, 3 HT, 4 MHT
`TrigObj_pt, eta, phi`	HLT object kinematics (e/gamma: the supercluster)
`TrigObj_l1pt, l1iso`	matched L1 seed in a cone (type -98 for e/gamma)
`TrigObj_filterBits`	one bit per HLT filter passed, per-type list in `triggerObjects_cff.py`; 32-bit up to 15_0_1, 64-bit from 15_0_2

```
# sketch of a NanoAOD trigger match (awkward)
to   = events.TrigObj
ph   = to[to.id == 22]
leg2 = ph[(ph.filterBits >> 17) & 1 == 1]
# unseeded R9 leg: always empty today
dr   = photons.metric_table(leg2)
ok   = ak.any(dr < 0.1, axis=2)
```

A leg-by-leg efficiency, a trigger-matched tag-and-probe, or a matching of both photons needs the bit of the last filter of each leg. The filter labels are not stored in NanoAOD: the bits are the only record. For the sub-leading leg of the diphoton triggers the bit is never set, so the match always fails.

The table is filled by TriggerObjectTableProducer from the unpacked MiniAOD trigger objects; an object is written only if it passes its type's selection string and sets at least one bit of that type.

Two e/gamma candidate collections at HLT: L1-seeded and unseeded

`hltEgammaCandidates` (seeded): superclusters only around L1 e/gamma candidates. Leading leg ($E_{\mathrm{T}} > 30\,\mathrm{GeV}$, L1 match) and almost every single-photon/electron path.

```
fragment.hltEgammaCandidates = cms.EDProducer(
  "EgammaHLTRecoEcalCandidateProducers",
  scHybridBarrelProducer = cms.InputTag(
    'hltParticleFlowSuperClusterECALL1Seeded',
    'hltParticleFlowSuperClusterECALBarrel' ), ...
```

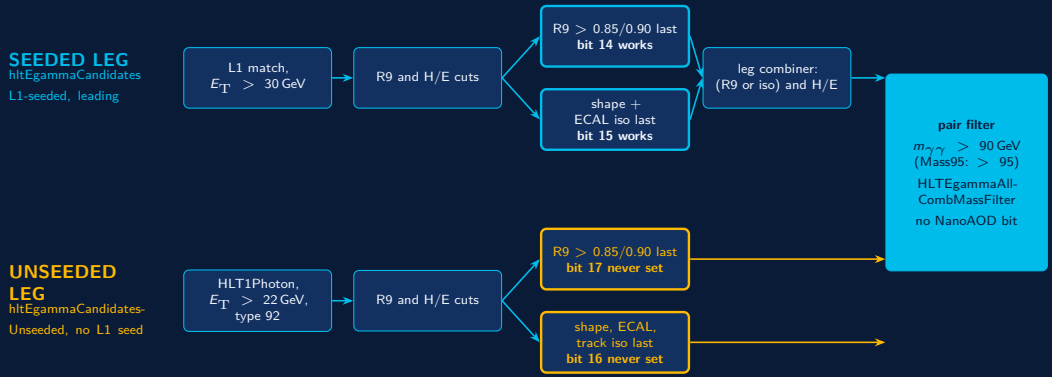
`hltEgammaCandidatesUnseeded`: superclusters over the whole ECAL, no L1 requirement. Sub-leading object of double-object paths ($E_{\mathrm{T}} > 22\,\mathrm{GeV}$ here).

```
fragment.hltEgammaCandidatesUnseeded = cms.EDProducer(
  "EgammaHLTRecoEcalCandidateProducers",
  scHybridBarrelProducer = cms.InputTag(
    'hltParticleFlowSuperClusterECALUnseeded',
    'hltParticleFlowSuperClusterECALBarrel' ), ...
```

Two producers, two collections, two distinct trigger objects even when they describe the same photon. The trigger summary, PAT and MiniAOD keep the collection name with each object; NanoAOD decides per object type which names it accepts, and that is where the photon table goes wrong.

Both producers from CMSSW_13_3_1_patch1 HLT_FULL_cff.py; the only difference is the supercluster input.

One path, two legs, four last filters



Gold: the unseeded last filters NanoAOD discards. Bits in v15 numbering; in v12 to v14 the same four last filters are bits 12, 13 (seeded) and 15, 14 (unseeded). Mass95 shares the whole sequence.

Each last filter is wrapped in cms.ignore(): it runs and saves its objects but cannot stop the path. The chain is identical (same MD5 of the ordered filter list) in all nineteen menu files checked.

From the HLT filter to `TrigObj_filterBits`: four hand-offs survive, the fifth drops the object

1 · HLT filters (saveTags): the unseeded last filters tag their objects with `hltEgammaCandidatesUnseeded`; the leg's first filter saves them with type 92, the later ones with 81.

2 · hltTriggerSummaryAOD: objects stored once per collection; every filter records the keys of its objects and the type it saved them with.

3 · patTrigger → slimmedPatTrigger (MiniAOD): the object keeps the label `hltEgammaCandidatesUnseeded::HLT`, both types (92, 81) and every filter label.

4 · unpackedPatTrigger (NanoAOD): filter labels unpacked; the object reaches the table producer intact.

5 · triggerObjectTable, Photon selection: `type(92) && pt > 15 && coll('hltEgammaCandidates')` is evaluated before the bits. The collection test fails, the object is dropped, bits 16 and 17 are never computed.

The Electron selection has accepted hltEgammaCandidatesUnseeded since June 2023 and its unseeded CaloIdL_MW bit is populated in production files: the objects are present up to the table producer.

One unseeded photon object through the Photon selection, test by test

1. `type(92)`: **true**. `hltDiEG22EtEta2p55UnseededFilter` saved it with triggerType 92; PAT kept 92 next to the 81 of the later filters.
2. `pt > 15`: **true**. The leg requires 22 GeV (18 GeV for Diphoton30_18).
3. `coll('hltEgammaCandidates')`: **false**. The label is `hltEgammaCandidatesUnseeded`; no wildcard, so an exact comparison.
4. `sel.match(obj)` is false: the bits are never evaluated, the object is never written, bits 16 and 17 are 0 in every event of every file.

The Electron selection in the same file:

```
# CMSSW_13_0_13 (v12)
sel = "type(92) && pt > 7
  && coll('hltEgammaCandidates')
  && filter('*PixelMatchFilter')"
# CMSSW_13_3_0 onward (v13), PR 41827
sel = "type(92) && pt > 7
  && (coll('hltEgammaCandidates')
      || coll('hltEgammaCandidatesUnseeded'))
  && (filter('*PixelMatchFilter')
      || filter('*PixelMatchUnseededFilter'))"
```

PR 41827 (merged 8 June 2023) added the unseeded collection to the Electron selection. The diphoton photon bits had been added four months earlier by PR 40711 without that change.

Backup frames give the code of every step: HLT1Photon, HLTGenericFilter, PATTriggerProducer, the slimmer, TriggerObjectTableProducer and TriggerObjectStandAlone::hasCollection.

Part II

Scope

Scope by NanoAOD version: present since the bits were added, still in master

NanoAOD	releases checked (tags and branch tips)	Photon sel	Electron sel	dead photon bits	affected
v9	CMSSW_10_6_X	seeded, pt>20, no bits	seeded	no photon TrigObj	no
v10, v11	CMSSW_12_4_X, CMSSW_12_6_X	seeded, pt>20, 12 bits	seeded	bits absent	no
v12	13_0_0 (pt>20), 13_0_1 to 13_0_17 (production 13_0_13)	seeded, 18 bits	seeded	14, 15, 17	yes
v13	13_3_0, 13_3_1_patch1 (production), 13_3_X tip	seeded, 18 bits	seeded + unseeded	14, 15, 17	yes
v14	14_0_0, 14_0_X, 14_1_0, 14_1_X, 14_2_0, 14_2_X	seeded, 18 bits	seeded + unseeded	14, 15, 17	yes
v15	15_0_2 to 15_0_15_patch4, 15_0_X, 15_1_0, 15_1_X (15_0_0, 15_0_1: 18 bits)	seeded, 21 bits	seeded + unseeded	16, 17, 19	yes
next	16_0_10, 16_0_X, 16_1_3, 16_1_X, master ed35654b1 (4 Nov 2026)	seeded, 21 bits	seeded + unseeded	16, 17, 19	yes

- Checked live on 4 November 2026: the Photon selection line is identical on master and on the tips of 14_1_X to 16_1_X and at the newest tags CMSSW_16_1_3 and CMSSW_16_0_10.
- No open pull request or issue mentions hltEgammaCandidatesUnseeded, triggerObjects_cff or photon trigger objects.
- Decimal values of the dead bits: 16384, 32768, 131072 (v12 to v14); 65536, 131072, 524288 (v15 and later).

NOTES.md sections 7.2 and 13.1.

Scope by HLT menu: nine years of menus, the same four filters on the same collections

menu (table name)	Mass90/95	Diphoton30_18	dead unseeded photon bits (v12–v14 / v15)
2018 GRun, /dev/CMSSW_10_6_0/GRun/V3	v13	NoPixelVeto v2	14, 15, 17 / 16, 17, 19
2022 frozen v1.2 (HLT_2022v12, Run3Summer22 MC)	v13	v2	14, 15, 17 / 16, 17, 19
2022 frozen v1.4 and v1.5, GRun 12_4 V171, GRun 12_6 V16	v15	v3	14, 15, 17 / 16, 17, 19
2023 frozen v1.2 (HLT_2023v12), GRun 13_0 V153	v17	v5 (renamed filters)	14, 15 / 16, 17
2023 development, 13_3 GRun V11 and HLT V19	v19	v7	14, 15 / 16, 17
2024 frozen v1.4 (HLT_2024v14), GRun 14_0 V182	v22	v10	14, 15 / 16, 17
2025 development (fork, May 2025), GRun 15_0 V62	v24	v12	16, 17
2025 frozen v1.3 (HLT_2025v13), GRun 15_0 V120	v26	v14	16, 17
2026 frozen v1.3 (HLT_2026v13, in master)	v29	v17	16, 17
2026 development, GRun 16_1 V16 (master)	v30	v18	16, 17

In all of these the two unseeded last filters carry `l1EGCand = hltEgammaCandidatesUnseeded` and `saveTags = True`. From 2023 the Diphoton30_18 filters were renamed, so its two bits (18, 19 in v15) match nothing in those menus: a separate stale-name problem.

Frozen menus used for MC: 2022v12 and 2022v14 (Summer22, Summer22EE), 2023v12 (Summer23, 23BPix), 2024v14 (Summer24), as reported by the production configurations.

Measured on twelve NanoAOD files: not one photon object with an unseeded bit since v12

NanoAOD	dataset (one file each)	events	Mass90	photons: seeded bits	unseeded bits	e unseeded	id 2	id 1515
v9	EGamma 2018D UL2018 NanoAODv9	300000	31771	no photon TrigObj	none defined	n/a	0	0
v10	EGamma 2022D PromptNanoAODv10	300000	46707	bits not defined	none defined	n/a	0	0
v11	EGamma 2022F PromptNanoAODv11	300000	41546	bits not defined	none defined	n/a	0	0
v12	GluGluHtoGG M125 Run3Summer22	12979	7725	15626 / 17978	**0 / 0 / 0**	n/a	0	0
v12	EGamma 2022D 22Sep2023	300000	46407	208527 / 255099	**0 / 0 / 0**	n/a	0	0
v13	GluGluHto2G M125 Run3Summer23	17760	10453	20865 / 24170	**0 / 0 / 0**	702	0	0
v13	EGamma 2022D 16Dec2023	300000	46794	207844 / 252774	**0 / 0 / 0**	29042	0	0
v14	EGamma0 2024D PromptReco	300000	35892	203620 / 252039	**0 / 0 / 0**	19444	0	0
v15	GluGluH-Hto2G M125 RunIII2024Summer24	4180	2406	4815 / 5588	**0 / 0 / 0**	205	0	22
v15	EGamma 2022D NanoAODv15	300000	48687	212270 / 258546	**0 / 0 / 0**	30451	0	430
v15	EGamma0 2024D MINIv6NANOv15	300000	30945	201628 / 248974	**0 / 0 / 0**	22980	0	254
133X	GluGluHToGG M125 powheg Run3Winter24NanoAOD	175654	101598	205090 / 239250	**0 / 0 / 0**	9860	0	0

- **Controls**: v9 has no photon objects; v10, v11 document no diphoton bits; the electron unseeded bit is absent in v12 and populated from v13.
- **Affected**: unseeded bits 0 in all 1.7 million events of v12 to v15 and Winter24, while the seeded bits are set on more objects than there are Mass90 events. No MET objects anywhere; BoostedTau only in v15.

Counts are photon objects (id 22) with the bit set, all events read. Seeded bits 12, 13 (v12–v14) / 14, 15 (v15); unseeded 14, 15, 17 / 16, 17, 19. lxplus, uproot (LCG_107), bit numbers parsed from each file. results/REPORT.md.

Scan of the whole table: the collection mismatch is unique to the Photon list

object	bits	never settable (collection or type)	fine/stale 2024	fine/stale 2025	stale in 2025, why
Electron	20	none	19 / 1	18 / 2	WPLoose (gone 2023), triple-electron (gone 2025)
Photon	21	**16, 17, 19 (unseeded) and 8 (hltHtE-cal800)**	15 / 3	15 / 3	bit 12 (gone 2022), 18 and 19 (renamed 2023)
Muon	14	none	14 / 0	12 / 2	mu-tau DeepTau overlaps (gone 2025)
Tau	31	none	31 / 0	30 / 1	DeepTau (gone 2025)
BoostedTau	3	v12 to v14: whole selection	2 / 1	2 / 1	2022 ParticleNetTauTau name
Jet	46	none	37 (+2 L1) / 7	34 (+2 L1) / 10	pixel-only HH4b (gone 2024), DeepCSV, DeepJet (gone 2025), 3-jet pt40/45
FatJet	7	none	4 / 3	4 / 3	2022 and 2023 soft-drop and ParticleNet names
HT, MHT	6, 2	none	3 (+1 L1) / 2, 2 / 0	same	two Run 2 L1 seed names
MET	0	no bits, skip flag on: never written	none	none	all versions since v9

- **Same mechanism** as the diphoton bits: only Photon bit 8, whose filter saves HT objects. Electron and Muon have no collection mismatch in any version or menu.
- **Stale names** are a documentation problem: one bit list serves 2022 to 2026, so renamed or retired filters stay documented but silently empty for later years.

scripts/deadbits.py, master configuration against the frozen 2024 v1.4 and 2025 v1.3 menus; raw outputs in scan/. The v12, v13 and v15 configurations give the same picture for their own bit lists.

Part III

Fix and validation

The fix: accept both e/gamma collections in the Photon selection

```
--- a/PhysicsTools/NanoAOD/python/triggerObjects_cff.py
+++ b/PhysicsTools/NanoAOD/python/triggerObjects_cff.py
@@ -87,7 +87,7 @@ triggerObjectTable = triggerObjectTableProducer.clone(
         Photon = cms.PSet(
             id = cms.int32(22),
-            sel = cms.string("type(92) && pt > 15 && coll('hltEgammaCandidates')"),
+            sel = cms.string("type(92) && pt > 15 && (coll('hltEgammaCandidates') || coll('hltEgammaCandidatesUnseeded
      '))"),
             l1seed = cms.string("type(-98)"),  l1deltaR = cms.double(0.3),
             skipObjectsNotPassingQualityBits = cms.bool(True),
```

Why this form: names exactly the two collections and copies the Electron selection of PR 41827. A wildcard `coll('hltEgammaCandidates*')` would also match, but would admit any future collection with that prefix.

What changes: unseeded objects enter the photon branch; with the skip flag only those that set a photon bit are written, and the only photon-bit filters on the unseeded collection are the three diphoton ones. A seeded twin in the same direction is merged with OR-ed bits.

What does not: bit numbers, documentation strings, the other object types and the event content. Existing files cannot be repaired: only a new NANO pass restores the sub-leading leg.

Commit aebf3eb64 on branch nano-photon-trigobj-unseeded (on top of upstream master ed35654b1) and 1699a39f7 on claude/fervent-lamport-3oiuh6, both on the fork santanumahata/cmssw. Line 83 in CMSSW_13_3_1_patch1, 86 in 15_0_15_patch4, 90 in master.

Fix test on real input: every Mass90 event gets its sub-leading-leg bits

NANO file (17760 events each)	Mass90	photons: seeded bits	photons: unseeded bits	Mass90 events with an unseeded bit	nTrigObj	size (bytes)
central v13 file	10453	20865 / 24170	0 / 0 / 0	0	5.287	29333713
v13 re-made, no fix	10453	20865 / 24170	0 / 0 / 0	0	5.287	29339920
v13 re-made, **with fix**	10453	20865 / 24170	**20461 / 18365 / 0**	**10453 (all)**	5.388	29363499
v15 re-made, no fix	10453	20865 / 24170	0 / 0 / 0	0	38.134	39947152
v15 re-made, **with fix**	10453	20865 / 24170	**20461 / 18365 / 0**	**10453 (all)**	38.225	39959608

- Without the fix the re-made file is **identical** to the central v13 file: same TrigObj (id, bits, p_{T}) in 17760 of 17760 events.
- The 21254 photon objects that gain an unseeded bit have p_{T} from **22.04 GeV** (median 58.30 GeV), none below the 22 GeV leg threshold.
- Every object type other than photons is unchanged in 17760 of 17760 events. Photon objects 24835 $\rightarrow$ 26639 (v13), 25188 $\rightarrow$ 26807 (v15); file size +0.08% (v13).
- The Diphoton30_18 unseeded bit stays 0: its filter name does not exist in the 2023 menu.

Unseeded bits 14 / 15 / 17 in v13, 16 / 17 / 19 in v15. Input: the MiniAOD parent of the central Run3Summer23 v13 GluGluHto2G M125 file. results/REPORT.md "Fix test"; commands in scripts/make_fixed_nano.sh.

Relval in CMSSW_16_1_3: the fix builds and runs; these workflows have no TrigObj

Clean CMSSW_16_1_3 versus the same release with commit aebf3eb64 cherry-picked (one changed line). Steps 1–2 ran once; steps 3–4 ran in both areas on the same `step2.root`.

workflow	content	TrigObj in NANO	NANO fix vs clean	DQM fix vs clean	step 3 log
12452.0	H125GGgluonfusion 2023, upgrade matrix, 10 generated events	0 in both	identical, 10/10	116785 histograms; 25 differ, all TimerService	clean
141.046	EGamma Run2023D run 369978, 100 events, 10 fire Mass90	0 in both	identical, 100/100	53846 histograms; 25 differ, all TimerService	clean

- No `Exception` and no `triggerObjectTable` line in any step-3 log; outputs identical apart from timing histograms.
- **The photon selection is not exercised**: the relval HLT summary is nearly empty. In 141.046 the reHLT summary is 1428 bytes per event against 34559 for the original 2023 HLT summary of the same file; `slimmedPatTrigger` is 372 bytes per event. Why was not investigated.
- The effect of the fix is therefore shown by the fix test on real MiniAOD input (previous frame).

results/REPORT.md "Relval in CMSSW_16_1_3"; scripts/relval_fix_check.sh and compare_dqm.py. 141.046 RAW is on tape only in DAS; one file was read from the IB EOS cache.

What analyses see, and what HiggsDNA does with the photon bits

- **Datasets**: every Run 3 NanoAOD from v12 on — 2022 to 2025 data (22Sep2023, 16Dec2023, prompt, v15 re-NANO) and the Summer22, 22EE, 23, 23BPix, 24 MC — plus any Run 2 input re-processed with v12 or later.
- **Frameworks**: NanoAOD-based photon matching finds no object for the sub-leading leg. MiniAOD-based flashgg reads the PAT objects directly and is unaffected.
- **HiggsDNA** (GitLab master d179305; same in ZhenyuDongLL/HiggsDNA-bbggrun3):
  - `trigger_match` (`higgs_dna/utils/misc_utils.py:306`) is called only in `workflows/dystudies.py:256`, for electrons (pdgid 11, bit 1 or 12).
  - No workflow tests a photon filter bit or a diphoton filter bit; `workflows/lowmass.py:641-642` only counts id-11 and id-22 objects.
  - The low-mass unseeded-leg efficiency comes from a precomputed `Lowmass_unseeded_leg_efficinecy.json`, whose derivation is not in the repository.
- **Elsewhere**: the unseeded filter names appear in 16 files outside cms-sw (analysis legends listing bits 16 and 17 as the legs, FLAF configurations, custom NanoAOD producers, a flashgg MiniAOD script).
- Existing files cannot be repaired: the unseeded objects were never written.

NOTES.md sections 9 and 13.5 (GitHub code search, default branches only).

Pull request status and the follow-ups kept out of the one-line fix

Pull request (prepared, not opened)

- Title: *NanoAOD: accept unseeded e/gamma trigger objects in the Photon TrigObj selection.*
- From santanumahata:nano-photon-trigobj-unseeded (aebf3eb64 on master ed35654b1) to cms-sw:master; xpog signature.
- Cites PR 40711 (bits added) and PR 41827 (electrons fixed).
- Backport to CMSSW_15_0_X if another production from that branch is planned.
- Both branches are on the fork; nothing pushed to cms-sw/cmssw; no pull request exists.
- Text: pr/PR_body.md of the notes branch.

Follow-ups, each a separate pull request

- 2023 names of the Diphoton30_18 filters as OR alternatives inside bits 18 and 19.
- Move bit 8 (hltHtEcal800) out of the Photon list, or drop it: its objects are HT objects.
- Say in the documentation which menus each bit covers; review the stale bits per year.
- Photon bits for the low-mass eta1p5, DiphotonMVA14p25, Diphoton15_1x EBEB and DoublePhoton paths, with the EGM POG and the Higgs PAG.
- Tell XPOG that MET trigger objects are never written and that the BoostedTau table was empty in v12–v14.

NOTES.md sections 8.2 and 13.6.

Summary

- The Photon trigger-object selection accepts only `hltEgammaCandidates`; the sub-leading leg of the Diphoton30_22 Mass90/Mass95 paths lives on `hltEgammaCandidatesUnseeded`. Its bits are **documented but can never be set**.
- Affects **NanoAOD v12, v13, v14, v15 and master**; all HLT menus 2018–2026. Measured on twelve files: 0 photon objects with an unseeded bit in 1.7 million events.
- **One-line fix**, the Electron construct of PR 41827. On real MiniAOD input it sets the unseeded bits in **all 10453** Mass90 events ($p_{\mathrm{T}} \geq 22.04\,\mathrm{GeV}$) and changes no other object.
- Relval in 16_1_3: builds and runs cleanly, outputs unchanged; those workflows carry no trigger objects.
- Next: open the pull request against cms-sw:master; four follow-ups kept separate.

Backup

Backup: the path and its sequence in the NanoAOD v13 release menu

```
# CMSSW_13_3_1_patch1, HLTrigger/Configuration/python/HLT_FULL_cff.py, line 119468 (path) and 118801 (sequence)
fragment.HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90_v19 = cms.Path(
    fragment.HLTBeginSequence + fragment.hltL1sSingleAndDoubleEGor
  + fragment.hltPreDiphoton3022R9IdORIsoCaloIdANDHER9IdMass90
  + fragment.HLTDiphoton30L22R9Id85b90eORIso60CaloId15b35eANDHE12R9Id50b80eSequence
  + fragment.hltDiEG22R9Id85b90eORIso60CaloId15b35eANDHE12R9Id50b80eMass90CombMassLastFilter
  + fragment.HLTEndSequence )
# the sequence, e/gamma modules only; cms.ignore(f) = f is run and saves its objects but cannot stop the path
hltEgammaCandidates > hltEGL1SingleAndDoubleEGOrFilter > hltEG30L1SingleAndDoubleEGOrEtFilter
  > hltEG30LR9Id50b80eR9IdFilter > hltEG30LHE12R9Id50b80eHEFilter
  > ignore(hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter)                    # bit 14
  > ignore(hltEG30LCaloId15b35eHE12R9Id50b80eClusterShapeFilter)
  > ignore(hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter)          # bit 15
  > hltEG30LRId85ORIso60CaloId15b35eANDHE12R9Id50b80eLegCombLastFilter
hltEgammaCandidatesUnseeded > hltDiEG22EtEta2p55UnseededFilter
  > hltDiEG22R9Id50b80eR9IdUnseededFilter > hltDiEG22HE12R9Id50b80eHEUnseededFilter
  > ignore(hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter)               # bit 17
  > ignore(hltEG22CaloId15b35eHE12R9Id50b80eClusterShapeUnseededFilter)
  > ignore(hltEG22Iso60CaloId15b35eHE12R9Id50b80eEcalIsoUnseededFilter)
  > ignore(hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter)  # bit 16
```

Variable producers between the filters are omitted. The path version changes from menu to menu (v13 in 2018, v29 in the frozen 2026 menu); the e/gamma filter chain does not (same MD5 in all nineteen menu files).

Backup: the two unseeded last filters, collection explicit in `l1EGCand`

```
# bit 17 (v12 to v14: bit 15), trimmed
fragment.hltEG22R9Id85b90eHE12R9Id50b80e
  R9UnseededLastFilter = cms.EDFilter(
  "HLTEgammaGenericFilter",
  saveTags = cms.bool( True ),
  candTag = cms.InputTag(
    "hltDiEG22HE12R9Id50b80eHEUnseededFilter" ),
  varTag = cms.InputTag( 'hltEgammaR9IDUnseeded', 'r95x5'
      ),
  lessThan = cms.bool( False ),
  useEt = cms.bool( False ),
  thrRegularEB = cms.vdouble( 0.85 ),
  thrRegularEE = cms.vdouble( 0.9 ),
  ncandcut = cms.int32( 1 ),
  l1EGCand = cms.InputTag(
    "hltEgammaCandidatesUnseeded" ) )
```

```
# bit 16 (v12 to v14: bit 14), trimmed
fragment.hltEG22Iso60CaloId15b35eHE12R9Id50b80e
  TrackIsoUnseededLastFilter = cms.EDFilter(
  "HLTEgammaGenericQuadraticFilter",
  saveTags = cms.bool( True ),
  candTag = cms.InputTag( "hltEG22Iso60CaloId15b35e
    HE12R9Id50b80eEcalIsoUnseededFilter" ),
  varTag = cms.InputTag( "hltEgammaHollowTrackIsoUnseeded
      " ),
  lessThan = cms.bool( True ),
  useEt = cms.bool( True ),
  thrRegularEB = cms.vdouble( 6.0 ),
  thrRegularEE = cms.vdouble( 6.0 ),
  thrOverEEB = cms.vdouble( 0.002 ),
  thrOverEEE = cms.vdouble( 0.002 ),
  ncandcut = cms.int32( 1 ),
  l1EGCand = cms.InputTag(
    "hltEgammaCandidatesUnseeded" ) )
```

candTag: previous filter of the chain; varTag: variable producer; `l1EGCand`: the collection the filter tags its saved objects with.

The seeded last filters are identical in structure with `l1EGCand = hltEgammaCandidates`.

Backup: why the unseeded objects carry type 92 (first filter of the leg)

```
# menu: the first unseeded filter
fragment.hltDiEG22EtEta2p55UnseededFilter =
  cms.EDFilter( "HLT1Photon",
  saveTags = cms.bool( True ),
  inputTag = cms.InputTag( "
      hltEgammaCandidatesUnseeded" ),
  triggerType = cms.int32( 92 ),
  MinPt = cms.double( 22.0 ),
  MaxEta = cms.double( 2.55 ),
  MinN = cms.int32( 2 ) )
```

```
// HLTrigger/HLTfilters/plugins/HLTSinglet.cc
template <typename T>
int getObjectType(const T&) { return 0; }
// no specialisation for RecoEcalCandidate
if (saveTags())
  filterproduct.addCollectionTag(inputTag_);
...
int tid = getObjectType<T>(*i);
if (tid == 0)
  tid = triggerType_;  // 92, TriggerCluster
filterproduct.addObject(tid, ref);
```

- Every candidate of `hltEgammaCandidatesUnseeded` with $E_{\mathrm{T}} > 22\,\mathrm{GeV}$ is saved with type 92 and that collection tag. On the seeded leg the L1-match and E_{T} filters save TriggerCluster as well.
- `type(92)` in NanoAOD is satisfied on both legs: the type is not what rejects the unseeded objects.

TriggerTypeDefs.h: TriggerPhoton $= 81$, TriggerCluster $= 92$, TriggerL1EG $= -98$.

Backup: what the later filters save (HLTGenericFilter)

```
// HLTrigger/Egamma/plugins/HLTGenericFilter.cc (master lines 28 to 32 and 139 to 159; same logic from 10_6_X on)
l1EGTag_ = iConfig.template getParameter<edm::InputTag>("l1EGCand");
...
if (saveTags()) {
  filterproduct.addCollectionTag(l1EGTag_);  // hltEgammaCandidatesUnseeded for the unseeded leg
}
// Set output format
int trigger_type = trigger::TriggerCluster;
if (saveTags())
  trigger_type = trigger::TriggerPhoton;          // 81 for every saved object
...
PrevFilterOutput->getObjects(TriggerCluster, recoCands);
if (recoCands.empty())
  PrevFilterOutput->getObjects(TriggerPhoton, recoCands);  // accepts either type from the previous filter
```

- **Collection**: the saved reference points into the `l1EGCand` collection; PAT turns it into the label `hltEgammaCandidatesUnseeded::HLT`.
- **Type**: these filters save 81; the object keeps the 92 of the first filter, because PAT attaches the type of every filter that accepted it.
- Same three statements in 10_6_X, 12_4_X, 12_6_X, 13_0_X, 13_3_X, 14_0_X, 15_0_X and master; HLTEgammaGenericQuadraticFilter.cc has the identical block (lines 118 to 135).

Backup: trigger summary and PAT keep collection label, all types, all filter labels

```
# menu: the summary producer
fragment.hltTriggerSummaryAOD = cms.EDProducer
      (
  "TriggerSummaryProducerAOD",
  throw = cms.bool( False ),
  processName = cms.string( "@" ),
  moduleLabelPatternsToMatch = cms.vstring( '
      hlt*' ),
  moduleLabelPatternsToSkip = cms.vstring( ) )
```

Every filter with saveTags contributes; objects are stored once per collection tag, each filter records the keys of its objects and the type it saved them with.

```
// PhysicsTools/PatAlgos/plugins/PATTriggerProducer.cc (master 680 to
       731)
for (size_t iK = 0; iK < keys.size(); ++iK) {
  filterLabels.insert({keys[iK], nameFilter});
  objectTypes.insert({keys[iK], types[iK]});  // per filter
}
...
triggerObject.setCollection(
    handleTriggerEvent->collectionTagEncoded(iC));
for (auto t = objectTypes.equal_range(iO); ...)
  triggerObject.addTriggerObjectType(t.first->second);
...
triggerObjectStandAlone.addFilterLabel(frange.first->second);
triggerObjectStandAlone.addPathName(iP->pathName, ...);
```

`setCollection` stores `hltEgammaCandidatesUnseeded::HLT` for the sub-leading leg; the object carries 92 and 81 and every filter label. patTrigger has `onlyStandAlone = False` and no excluded collections: nothing from the unseeded collection is lost here.

Backup: MiniAOD slims the objects but keeps the collection name

```
# selectedPatTrigger_cfi.py
selectedPatTrigger = cms.EDFilter(
  "PATTriggerObjectStandAloneSelector",
  src = cms.InputTag("patTrigger"),
  cut = cms.string("!filterLabels.empty()")
)
```

Keeps every object accepted by at least one saveTags filter.

```
# slimmedPatTrigger_cfi.py
slimmedPatTrigger = cms.EDProducer(
  "PATTriggerObjectStandAloneSlimmer",
  src = cms.InputTag("selectedPatTrigger"),
  triggerResults =
    cms.InputTag( 'TriggerResults::HLT' ),
  packFilterLabels = cms.bool(True),
  packP4 = cms.bool(True),
)
```

Packs four-vector and filter labels; collection string and types stored as they are.

```
# NanoAOD triggerObjects_cff.py, 8 to 12
unpackedPatTrigger = cms.EDProducer(
  "PATTriggerObjectStandAloneUnpacker",
  patTriggerObjectsStandAlone =
    cms.InputTag('slimmedPatTrigger'),
  triggerResults =
    cms.InputTag('TriggerResults::HLT'),
  unpackFilterLabels = cms.bool(True)
)
```

Restores the labels so that `filter('...')` can be evaluated.

From MiniAOD on the object is a pat::TriggerObjectStandAlone: packed four-vector, collection string, type list, packed filter labels, path names.

Backup: the NanoAOD Photon selection — one collection in `sel`, three unseeded filters in the bit list

```
# PhysicsTools/NanoAOD/python/triggerObjects_cff.py, master lines 88 to 116 (v13: 81 to 108), abbreviated
Photon = cms.PSet(
  id = cms.int32(22),
  sel = cms.string("type(92) && pt > 15 && coll('hltEgammaCandidates')"),
  l1seed = cms.string("type(-98)"),  l1deltaR = cms.double(0.3),
  skipObjectsNotPassingQualityBits = cms.bool(True),
  qualityBits = cms.VPSet(
    mksel("filter('hltEG33L1EG26HEFilter')","hltEG33L1EG26HEFilter"),                        # bit 0
    ...  # bits 1 to 13: HE filters, hltHtEcal800, EB tight-ID filters, Mu17_Photon30 photon leg
    mksel("filter('hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter')", ...),                   # 14
    mksel("filter('hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter')", ...),         # 15
    mksel("filter('hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter')", ...), # 16
    mksel("filter('hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter')", ...),              # 17
    mksel("filter('hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter')", ...), # 18
    mksel("filter('hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter')", ...), # 19
    mksel("filter('hltEG*L1VBFLooseIsoEGHEFilter')", "hltEG*L1VBFLooseIsoEGHEFilter"),        # 20
  )
),
```

- `mksel` turns a filter name into `filter('name')`, a list into an OR, a tuple into an AND; bit i is the position in the VPSet.
- The contradiction is inside this one PSet: `sel` admits only `hltEgammaCandidates`, while bits 16, 17, 19 name filters that run only on `hltEgammaCandidatesUnseeded`. No era modifier touches this block.
- In v12 to v14 the list has 18 entries (diphoton filters at 12–17); the 21-entry list and 64-bit filterBits arrived in CMSSW_15_0_2 (PR 47601, backport of PR 47594).

Backup: the table producer — selection first, bits second, duplicates merged last

```
// TriggerObjectTableProducer.cc, constructor, lines 105
      to 118
for (size_t i = 0; i != cfg.size(); ++i) {
  unsigned int bit = i;
  if (cfg[i].existsAs<unsigned int>("bit"))
    bit = cfg[i].getParameter<unsigned int>("bit");
  assert(!bits[bit] && "...inserted twice");
  assert(bit < 64 && "...on 64 bits");
  func << std::to_string(1UL << bit) << "*("
       << cfg[i].getParameter<std::string>("selection")
      << ")";
}
qualityBits = StringObjectFunction<...>(func.str());
```

```
// produce(), master lines 151 to 176 (abridged)
for (const auto &obj : trigObjs) {
  for (const auto &sel : sels_) {
    if (sel.match(obj)) {          // fails here
      bits = sel.qualityBits(obj);
      if (skip ? (bits != 0) : true)
        selected.emplace_back(&obj, &sel);
    }
  }
}
// Self-cleaning
for (i ...) for (j < i ...)
  if (sel.id == sel2.id && deltaR2(obj, obj2) < 1e-6) {
    bits_of(obj2) |= bits_of(obj);
    selected.erase(selected.begin() + i); i--;
  }
```

- **Order of evaluation is the whole story**: an object that fails `sel` never reaches the bits, never enters `selected`, and is never seen by the merging loop.
- Since 13_0_X an object is tested against every selection (it can be both id 11 and id 22); 12_4_X had a `break` after the first match.

Backup: the producer changed three times since 2022; selection-before-bits never did

releases (NanoAOD)	filterBits	bit limit	duplicate merging	selections per object
12_4_X, 12_6_X (v10, v11)	Int_t	bit < 31	same id, same p_{T}, same direction	first match only (break)
13_0_X to 14_2_X, 15_0_0, 15_0_1 (v12–v14)	Int_t	bit < 31	same id, $\lvert\Delta p_{\mathrm{T}}\rvert < 10^{-6}$, $\Delta R^2 < 10^{-6}$ (p_{T} test dropped in 15_0_0)	every matching selection
15_0_2 onward, master (v15, next)	uint64	bit < 64	same id, $\Delta R^2 < 10^{-6}$	every matching selection

- In every version `sel.match(obj)` is tested first; only a passing object gets its bits.
- The merging rule only matters after the fix: a seeded and an unseeded object describing the same photon merge on direction alone with the master rule.

Checked at the tags 14_1_0, 14_2_0, 15_0_0, 15_0_1, 15_0_2, 15_0_5, 15_0_10, 15_0_13, 15_0_15, 15_0_15_patch4, 15_1_0, 16_0_0 and the tips of 12_4_X, 13_0_X, 14_0_X, 15_0_X. The 64-bit change matches PR 47594 / 47601, merged 18 March 2025.

Backup: how `coll('hltEgammaCandidates')` is evaluated — no star, exact label comparison

```
// DataFormats/PatCandidates/src/TriggerObjectStandAlone.
      cc
bool TriggerObjectStandAlone::hasCollection(
    const std::string &collName) const {
  // Move to wild-card parser, if needed only
  if (collName.find(wildcard_) != std::string::npos) {
    if (hasAnyName(collName, {collection()}))
      return true;
    ...  // label / instance / process with wildcards
    return false;
  }
  // Use parent class's method otherwise
  return TriggerObject::hasCollection(collName);
}
```

```
// DataFormats/PatCandidates/src/TriggerObject.cc, 54 to
      71
bool TriggerObject::hasCollection(
    const std::string& collName) const {
  if (collName == collection_) return true;
  const edm::InputTag collectionTag(collection_);
  const edm::InputTag collTag(collName);
  if (collTag.process().empty()) {
    if ((collTag.instance().empty()
        && collectionTag.instance().empty())
        || collTag.instance() == collectionTag.instance()
      ) {
      return (collTag.label() == collectionTag.label());
    }
  }
  return false;
}
```

For the unseeded object: `collName = hltEgammaCandidates`, `collection_ = hltEgammaCandidatesUnseeded::HLT`. No star, so the base method runs; the labels differ; the result is false. `filter('...')` behaves the same way through `hasFilterOrCondition`.

Backup: six other explanations, and the evidence that rules each one out

hypothesis	what the code and the menus show	verdict
Filter names misspelt or outdated	All four Diphoton30_22 names defined with saveTags = True in the 2018, 2022 v1.2/v1.4/v1.5, 2023, 2024, 2025, 2026 menus and every development menu checked (19 files); chain identical	ruled out
Unseeded objects never reach MiniAOD	patTrigger excludes no collection; selectedPatTrigger keeps objects with any filter label; the Electron selection consumes the unseeded collection in production since June 2023	ruled out
Wrong type for `type(92)`	HLT1Photon saves them with triggerType 92; the later filters add 81; PAT keeps both	ruled out
The p_{T} threshold removes them	sel requires pt $>$ 15; the leg requires 22 GeV (18 GeV for Diphoton30_18)	ruled out
An era modifier changes the Photon block	Modifiers act on Muon, Tau, Jet (Run 2 eras) and the prefiring producer; none on Photon	ruled out
Specific to the v13 release	13_3_1_patch1 and the 13_3_X tip are byte-identical; 13_0_X to master carry the same selection	ruled out
The Photon selection rejects the collection	`coll('hltEgammaCandidates')`, exact comparison, before the bits	**the cause**

NOTES.md section 7.1.

Backup: why not a wildcard, and exactly which objects the fix lets through

```
// TriggerObjectStandAlone::hasAnyName, lines 80 to 119
       (abridged)
boost::split(parts, name, is_any_of("*"),
      token_compress_on);
for (name in nameVec) {
  index = 0;
  for (part in parts) {
    if (part.empty()) continue;
    index = name.find(part, index);
    if (index == npos ||
        (part is first && index > 0)) { failed; break;
      }
    index += part.length();
  }
  // Failed, if end of name not reached
  if (index < name.length() && !parts.back().empty())
    failed = true;
}
```

unseeded filter that is a photon bit	bit	menus
hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter	16	2018–2026
hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter	17	2018–2026
hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter	19	2018, 2022

The only photon-list entries whose filters run on the unseeded collection (nine menus checked; the wildcard bit 20 matches only the seeded `hltEG22L1VBFLooseIsoEGHEFilter`). Other unseeded e/gamma objects set no photon bit and are dropped by the skip flag.

`coll('hltEgammaCandidates*')` would match both collections. The anchoring rule is also what kills the v12–v14 BoostedTau pattern (no trailing star, name continues).

Backup: the breadth scan — every filter bit of every object type against each menu

```
# scripts/deadbits.py (notes repository), the core
def glob_re(pat):  # NanoAOD wildcard semantics
  parts = pat.split("*")
  return re.compile("^" + ".*".join(re.escape(p) for p in parts) + "$")
def collections_of(mods, name):
  # EDProducer labels a filter saves its objects from
  typ, body = mods[name]
  for p in ("l1EGCand", "inputTag", "CandTag", "inputCandCollection", "originTag1", ...):
    for lab in tag_labels(body, p):
      if mods[lab][0] == "EDProducer": direct.add(lab)
      elif mods[lab][0] == "EDFilter": follow.append(lab)
  return direct or union(collections_of(mods, f) for f in follow)
status = OK if any matched filter's collection passes coll()
       else COLL if filters match but none passes
       else STALE if no module matches the pattern
```

- **Inputs**: configs master, 15_0_X (v15), 13_3_1_patch1 (v13), 13_0_13 (v12); frozen menus 2022 v1.2 and v1.4, 2023 v1.2, 2024 v1.4, 2025 v1.3, 2026 v1.3 and the GRun menus of 13_0_X, 14_0_X, 15_0_X, 16_1_X.
- **Emulates** mksel, the anchored star matching of hasAnyName, the collection each matched filter saves and the `coll()` terms. The type test is not emulated; Tau's `coll('*Tau*')` is treated as non-restrictive.

Backup: three more entries that can never be filled

Photon bit 8: hltHtEcal800

```
fragment.hltHtEcal800 = cms.EDFilter
      (
  "HLTHtMhtFilter",
  saveTags = cms.bool( True ),
  htLabels = cms.VInputTag( '
      hltHtMhtEcal' ),
  minHt = cms.vdouble( 800.0 ), ... )

// HLTHtMhtFilter.cc, 104 to 105
addObject(trigger::TriggerTHT, htref)
      ;
addObject(trigger::TriggerMHT,
      mhtref);
```

HT objects (types 89, 90): never type 92, never an e/gamma collection. Dead since v10.

MET: no bits, skip flag on

```
MET = cms.PSet(
  id = cms.int32(2),
  sel = cms.string("type(87) && pt >
       30
    && coll('hltPFMETProducer')"),
  skipObjectsNotPassingQualityBits
    = cms.bool(True),
  qualityBits = cms.VPSet()
)
```

The bit function is the constant 0, so every MET object is dropped: TrigObj has never contained MET, although the documentation lists id 2.

BoostedTau in v12 to v14

```
# 13_0_X to 15_0_1
sel = "type(85) && pt > 120
  && coll('hltAK8PFJetsCorrected')
  && filter('hltAK8SinglePFJets*
     SoftDropMass40*
      ParticleNetTauTau')"
# menus: 2022 ...
      ParticleNetTauTau0p30
#        2023+ ...PNetTauTauTag0p03
# 15_0_2: filter('*TauTau*')
```

End-anchored pattern, no menu name matches; jets come from hltAK8PFJets230SoftDropMass40. Relaxed in 15_0_2.

Side findings of the scan, for XPOG; none changes the photon fix. Measured: no id-2 object in any file; BoostedTau objects only in the v15 files (22, 430, 254).

Backup: stale bits by year — documented filters that no longer exist in that year's menu

object	bit	filter pattern (master list)	exists in menus	empty for data of
Electron	2	`hltEle*WPLoose*TrackIsoFilter`	2022	2023 onward
Electron	8	`hltEle*Ele*Ele*CaloIdLTrackIdLDphiLeg*Filter` (3e)	2022 to 2024	2025 onward
Photon	12	`hltEG120EBTightIDTightIsoTrackIsoFilter`	2018, early 2022	2022 v1.4 onward
Photon	18, 19	Diphoton30_18 filters (hltEG30Iso60... and hltEG18TrackIso60...)	2018, 2022	2023 onward (renamed)
Muon	2, 6	`*OverlapFilterIsoMu*PFTau*` (DeepTau mu-tau overlap)	2022 to 2024	2025 onward
Tau	3	`*DeepTau*`	2022 to 2024	2025 onward
BoostedTau	0	`hltAK8SinglePFJets*SoftDropMass*ParticleNetTauTau0p30`	2022	2023 onward
Jet	0, 1, 6, 22, 27	pixel-only PFCentralJetTightID (HH to 4b)	2022, 2023	2024 onward
Jet	8, 9, 11, 24, 25	3PFCentralJetTightIDPt40/45, BTagCaloDeepCSVp17Double, BTagPFDeepJet1p5Single/4p5Triple	2022 to 2023/2024	2023, 2024 or 2025 onward
FatJet	3, 4, 5	SoftDropMass40BTagParticleNetBB0p35, AK8DoublePFJetSDModMass30/50	2022 (2023 for 4, 5)	2023 or 2024 onward
HT	0, 1	Run 2 L1 seed names	Run 2 only	all Run 3

Bits added for newer filters are empty for earlier years in the same way. This is the price of one bit list for five years of menus, not a code defect; the photon bits are the only ones no menu can cure.

Read off the scanner output for the master configuration against the 2022 to 2026 frozen and development menus.

Backup: photon filter bits 0 to 10 (master numbering)

bit v15	bit v12–v14	filter	objects come from	filter exists in
0–7	0–7	`hltEG33L1EG26HEFilter`, `hltEG50/75/90/120/150/175/200HEFilter`	hltEgammaCandidates	2018 to 2026
8	8	`hltHtEcal800`	**hltHtMhtEcal (HT object): never a photon**	2018 to 2026
9	none	`hltEG45EBTightIDTightIsoTrackIsoFilter`	hltEgammaCandidates	2024 to 2026
10	none	`hltEG50EBTightIDTightIsoTrackIsoFilter`	hltEgammaCandidates	2023 to 2026

Bits 0 to 7: single-photon H/E filters of the HLT_Photon paths; 9 to 12: barrel tight-ID isolated photon filters; 13: photon leg of Mu17_Photon30. Only bit 8 is misplaced in this half of the list.

triggerObjects_cff.py at master ed35654b1 (21 entries) and CMSSW_13_3_1_patch1 (18 entries); "objects come from" is the l1EGCand (or htLabels) of the module in the menu.

Backup: photon filter bits 11 to 20, with the three dead unseeded entries

v15	v12–v14	filter	objects come from	filter exists in
11	9	hltEG110EBTightIDTightIsoTrackIsoFilter	hltEgammaCandidates	2018 to 2026
12	10	hltEG12EBTightIDTightIsoTrackIsoFilter	hltEgammaCandidates	2018, 2022 v1.2 only
13	11	hltMu17Photon30IsoCaloIdPhotonlegTrackIsoFilter	hltEgammaCandidates	2018 to 2026
14	12	hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter	hltEgammaCandidates	2018 to 2026
15	13	hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter	hltEgammaCandidates	2018 to 2026
16	14	hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter	**hltEgammaCandidatesUnseeded, never set**	2018 to 2026
17	15	hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter	**hltEgammaCandidatesUnseeded, never set**	2018 to 2026
18	16	hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter	hltEgammaCandidates	2018, 2022 (renamed 2023)
19	17	hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter	**hltEgammaCandidatesUnseeded, never set**	2018, 2022 (renamed 2023)
20	none	hltEG*L1VBFLooseIsoEGHEFilter (only hltEG22L1VBFLooseIsoEGHEFilter)	hltEgammaCandidates	2023 to 2026

From 2023 the Diphoton30_18 filters are called hltEG30Iso60b40eCaloId11b32eHE10b9eR9Id50b90eEcalIsoFilter and

hltDiEG18TrackIso60Iso60b40eCaloId11b32eHE10b9eR9Id50b90eTrackIsoUnseededFilter, so bits 18 and 19 match nothing for 2023 and later data.

Backup: how the defect was born, survived and was found

- **7–8 Feb 2023**: PR 40711 adds the six diphoton photon bits for the next NanoAOD production. The Photon selection is not touched: the three unseeded bits are dead from day one (CMSSW_13_0_0; v12 made with 13_0_13).
- **31 May – 8 Jun 2023**: PR 41827 gives the Electron selection the unseeded collection and the CaloIdL_MW unseeded bit. Photons are not revisited. In CMSSW_13_3_0, so v13 has it for electrons only.
- **Nov – Dec 2023**: 13_3_X backports for v13 (43292 from 43247, 43286, 43470 and others); CMSSW_13_3_1_patch1 produces v13 for Summer22, 22EE, 23, 23BPix MC and the 16Dec2023 2022 data.
- **13–18 Mar 2025**: PR 47594 / 47601 add hltEG45 and hltEG50 EB bits and the VBF wildcard; filterBits becomes 64-bit (CMSSW_15_0_2). The unseeded bits move to 16, 17, 19, still dead. v15 inherits them.
- **4 Nov 2026**: cause located, confirmed on master and the 16_x branches, whole-table scan, fix committed, ten files measured on lxplus.
- **5 Nov 2026**: two more files, fix test on real MiniAOD input in 13_3_1_patch1 and 15_0_15_patch4, relval in 16_1_3.

Dates are the creation and merge dates from the GitHub search API. Between CMSSW_13_0_0 and 13_0_1 the photon pt threshold moved from 20 to 15 GeV; that change did not touch the collection test.

Backup: NanoAOD versions, releases and the v13 production recipe

NanoAOD	release (evidence as in NOTES.md 3.1)
v9	CMSSW_10_6_X (era modifier run2_nanoAOD_106Xv2)
v10	CMSSW_12_4_X (run3_nanoAOD_124)
v11, v11_1	CMSSW_12_6_X (reported)
v12	CMSSW_13_0_X, production 13_0_13 (reported)
v13	CMSSW_13_3_X, production 13_3_1_patch1
v14	CMSSW_14_0_X (reported)
v15	CMSSW_15_0_X: $\geq$15_0_2 for 2024; 15_0_15_patch4 for 2022/2023 inputs (run3_nanoAOD_pre142X)

v13 MC campaign: global tag	era
Run3Summer22: 133X_mcRun3_2022_realistic_ForNanov13_v1	Run3
Run3Summer22EE: 133X_mcRun3_2022_realistic_postEE_ForNanov13_v1	Run3
Run3Summer23: 133X_mcRun3_2023_realistic_ForNanov13_v1	Run3_2023
Run3Summer23BPix: 133X_mcRun3_2023_realistic_postBPix_ForNanov13_v2	Run3_2023

Input: 13_0_X MiniAODv4. Data: 2022 re-processed from 22Sep2023 MiniAOD as 16Dec2023 (v13); 2023 data stays at v12.

The version of a NanoAOD file is set by the release it was made in plus the era: there is no version-specific switch in the code. The file itself tells: 12 photon bits (v10, v11), 18 (v12 to v14), 21 with a 64-bit `filterBits` (v15).

NOTES.md sections 3.1 to 3.4; v13 command shape from public configurations that mirror the McM setup.

Backup: how the twelve files were measured

- lxplus, Python with uproot and awkward from LCG_107, files read through `root://cms-xrd-global.cern.ch/`; one file per dataset, the first `dasgoclient` returns; up to 300000 events per file.
- Bit numbers **parsed from each file's own** `TrigObj_filterBits` documentation, never hard-coded.
- Counted per file: photon objects with each seeded and unseeded bit (all events and Mass90-fired events), electron objects with the CaloIdL_MW-unseeded bit, objects per TrigObj id, events with any photon carrying an unseeded bit (with run, lumi, event of the first 50).
- File layout cross-check: 12 photon bits (v10, v11), 18 (v12 to v14), 21 and >u8 (v15).
- Documentation curiosities seen in the files: the v10 file lists 11 photon filters (no Mu17_Photon30 leg); the v11 file names bits 5 and 6 both hltEG150HEFilter; the Winter24 file carries 133X and the 18-bit layout.
- Not covered: NanoAOD v14 (14_0_X) MC is not in DAS.

scripts/measure_trigobj.py; one JSON per file under results/; results/REPORT.md.

Backup: the fix test, exactly as run (scripts/make_fixed_nano.sh)

```
INPUT=root://cms-xrd-global.cern.ch//store/mc/Run3Summer23MiniAODv4/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-
      pythia8/
      MINIAODSIM/130X_mcRun3_2023_realistic_v15_ext1-v4/2820000/74338d47-0b38-4e0b-83a2-ad85c7f4e533.root   # parent of
       central v13 file
# v13: CMSSW_13_3_1_patch1, era Run3_2023,                        GT 133X_mcRun3_2023_realistic_ForNanov13_v1
# v15: CMSSW_15_0_15_patch4, era Run3_2023,run3_nanoAOD_pre142X,  GT 150X_mcRun3_2023_realistic_v1
scram project CMSSW $RELEASE; cd $RELEASE/src; eval $(scramv1 runtime -sh)
git cms-addpkg PhysicsTools/NanoAOD; scram b -j 8
cmsDriver.py nano_nofix --step NANO --mc --scenario pp --nThreads 4 --eventcontent NANOAODSIM --datatier NANOAODSIM \
    --conditions $GT --era $ERA --filein $INPUT --fileout file:nano_nofix.root -n -1
sed -i "s#type(92) && pt > 15 && coll('hltEgammaCandidates')\"#type(92) \&\& pt > 15 \&\& (coll('hltEgammaCandidates')
      || coll('hltEgammaCandidatesUnseeded'))\"#" \
    PhysicsTools/NanoAOD/python/triggerObjects_cff.py
git diff --numstat          # must be exactly "1 1"
scram b -j 8
cmsDriver.py nano_fix  ... same options ... --fileout file:nano_fix.root
python3 measure_trigobj.py v13_nofix ... nano_nofix.root nano_nofix.json <central v13 file>   # event-by-event
      comparison
python3 measure_trigobj.py v13_fix   ... nano_fix.root   nano_fix.json   nano_nofix.root
```

- The MiniAOD dataset first proposed (`...realistic_v14-v2/MINIAODSIM`) has 0 files in DAS; the DAS parent of the central v13 file was used instead, so the no-fix output can be compared with the central file event by event.
- The v15 global tag is the processing string of `Run3Summer23NanoAODv15` in DAS (no GluGluH dataset exists there).
- The changed line is 83 in 13_3_1_patch1 and 86 in 15_0_15_patch4. One v13 `nano_fix` run was terminated externally at 8532 events and rerun with the same configuration.

Backup: the relval check in detail

- **Areas**: `clean` = CMSSW_16_1_3; `fix` = the same release + `git cms-addpkg PhysicsTools/NanoAOD` + `git cherry-pick aebf3eb64`; `git diff CMSSW_16_1_3 -numstat` = one line. Cherry-pick instead of `git cms-merge-topic`, because the topic branch sits on master, not on the 16_1_X release.
- **12452.0**: not in the default matrix of 16_1_3 (`-w upgrade` needed); its RelVal GEN-SIM input returned no file at T2_CH_CERN, so step 1 generated the 10 events.
- **141.046**: run 369978 RAW is on tape only in DAS (T0_CH_CERN_Tape, T1_US_FNAL_Tape); of 321 files listed, one is in the IB EOS cache and was used for step 2 (100 events). Steps 3–4 from `runTheMatrix -dryRun`.
- Step-3 NANO is EDM (NANOEDMAOD); converted to flat NANO with `cmsDriver.py -step NONE` before measuring.
- **Per-event sizes** (`edmEventSize -v`): 141.046 `hltTriggerSummaryAOD__reHLT` 1427.78 B vs `hltTriggerSummaryAOD__HLT` 34558.6 B; `slimmedPatTrigger__reRECO` 371.96 B; 12452.0 `hltTriggerSummaryAOD__HLT` 1313 B.

scripts/relval_fix_check.sh, scripts/compare_dqm.py; JSONs in results/relval_16_1_3/.

Backup: counting the new bits on any NanoAOD file

```
# count photon objects with the unseeded bits, per path
import uproot, awkward as ak, numpy as np
t = uproot.open("nano.root")["Events"]
path = "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90"
a = t.arrays(["TrigObj_id", "TrigObj_filterBits", path])
ph    = a["TrigObj_id"] == 22
bits  = ak.values_astype(a["TrigObj_filterBits"][ph], np.uint64)
fired = a[path]
for bit in (16, 17):   # 14, 15 for NanoAOD v12 to v14
  has = ak.any(((bits >> np.uint64(bit)) & np.uint64(1)) == 1, axis=1)
  print(bit, int(np.sum(has & fired)), "of", int(np.sum(fired)))
```

- Before the fix both counts are 0 by construction. In the fix test every one of the 10453 Mass90 events has a photon with at least one of the two bits.
- For v12 to v14 files the loop runs over bits 14 and 15 and `filterBits` is 32-bit; the uint64 cast is harmless.

NOTES.md section 8.1; the full measurement is scripts/measure_trigobj.py.

Backup: what the trigger-object rows look like, today and with the fix

```
# schematic TrigObj rows of one diphoton-triggered event (illustration of the structure, not real data)
today (v15):
 id  pt     filterBits         meaning
 22  41.7   0x4000 (bit 14)    leading leg, R9 last
 -- no row for the 26 GeV photon --
with the fix:
 22  41.7   0x4000 (bit 14)    leading leg, R9 last
 22  26.3   0x20000 (bit 17)   unseeded leg, R9 last
```

- A leading photon that also passes the unseeded leg merges with its unseeded twin (same direction) and carries both bits; a sub-leading photon appears as a new row.
- The rows are made up to show the structure; p_{T} values and row counts differ event by event. The measured effect is in the fix-test table: photon objects 24835 $\rightarrow$ 26639 for 17760 events (v13).

Backup: where everything is

- **Notes branch**: https://github.com/santanumahata/cmssw/tree/nanoaod-trigobj-notes
  - NOTES.md: the knowledge base (sections 0 to 16).
  - results/: REPORT.md, one JSON per measured file, fix_test/, relval_16_1_3/.
  - scripts/: measure_trigobj.py, make_fixed_nano.sh, relval_fix_check.sh, compare_dqm.py, deadbits.py and the menu scanners.
  - scan/: raw scanner outputs; pr/: patch and PR text; report/: issue report (PDF); slides/: the HTML decks and this Beamer deck.
- **Fix branch**: https://github.com/santanumahata/cmssw/tree/nano-photon-trigobj-unseeded (aebf3eb64).
- **Open the pull request** (creates nothing by itself): https://github.com/cms-sw/cmssw/compare/master...santanumahata:cmssw:nano-photon-trigobj-unseeded

Backup: code

Backup code: verify_nanoaod_files.py (1/6)

scripts/verify_nanoaod_files.py, lines 1 to 30

```
#!/usr/bin/env python3
"""Empirical check of the dead unseeded-leg photon TrigObj bits in CMS NanoAOD files.

usage:
  python3 verify_nanoaod_files.py --label v13 --files root://cms-xrd-global.cern.ch//store/... [more files]
                                  [--max-events 300000] [--out results_v13.json]

For every file it reads the TrigObj_filterBits documentation (branch title) to find the bit numbers
of the relevant filters in THAT file (so v12 to v14 and v15 numbering are both handled), then counts:
  - events firing the diphoton paths (and the DoubleEle CaloIdL_MW paths, electron control),
  - photon trigger objects (id 22) with the seeded last-filter bits (expected > 0),
  - photon trigger objects with the unseeded last-filter bits (the claim: always 0 before the fix),
  - photon objects with the hltHtEcal800 bit (claim: always 0),
  - electron objects (id 11) with the "CaloIdL_MW unseeded" bit (control: > 0 from v13 on),
  - trigger objects per id (MET id 2 expected 0 in every version; BoostedTau id 1515 expected 0 in v12 to v14).
Needs python3 with uproot, awkward, numpy (LCG view or a CMSSW environment) and xrootd access for root:// files.
"""
import argparse, json, re, sys, time
import numpy as np
import awkward as ak
import uproot

UNSEEDED = {
    "TrackIsoUnseededLast": "hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter",
    "R9UnseededLast": "hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter",
    "EG18TrackIsoUnseeded": "hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter",
}
SEEDED = {
    "R9IdLast": "hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter",
    "EcalIsoLast": "hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter",
```

Backup code: verify_nanoaod_files.py (2/6)

scripts/verify_nanoaod_files.py, lines 31 to 60

```
31      "EG30EcalIso3018": "hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter",
32  }
33  HT_BIT = "hltHtEcal800"
34  ELE_UNSEEDED_DOC = "CaloIdL_MW unseeded"
35  PATHS = [
36      "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90",
37      "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass95",
38      "HLT_Diphoton30_18_R9IdL_AND_HE_AND_IsoCaloId",
39      "HLT_Diphoton30_18_R9IdL_AND_HE_AND_IsoCaloId_Mass55",
40      "HLT_DoubleEle33_CaloIdL_MW",
41      "HLT_DoubleEle25_CaloIdL_MW",
42  ]
43  IDS = {2: "MET", 11: "Electron", 22: "Photon", 13: "Muon", 15: "Tau", 1515: "BoostedTau", 1: "Jet", 6: "FatJet", 3: "HT", 4: "MHT"}
44
45
46  def parse_bitdoc(title):
47      """'extra bits ...: 0 => a, 1 => b for Electron; 0 => c for Photon; ...' -> {object: {bit: doc}}"""
48      out = {}
49      body = title.split(":", 1)[1] if ":" in title else title
50      for chunk in body.split("; "):
51          m = re.search(r"\s+for\s+(\w+)\s*$", chunk.strip())
52          if not m:
53              continue
54          obj, entries = m.group(1), chunk.strip()[: m.start()]
55          d = {}
56          for e in re.finditer(r"(\d+)\s*=>\s*(.*?)(?=,\s*\d+\s*=>|$)", entries):
57              d[int(e.group(1))] = e.group(2).strip()
58          out[obj] = d
59      return out
```

Backup code: verify_nanoaod_files.py (3/6)

scripts/verify_nanoaod_files.py, lines 61 to 90

```
61
62  def find_bit(docs, needle):
63      for bit, doc in docs.items():
64          if needle in doc:
65              return bit
66      return None
67
68
69  def has_bit(bits, bit):
70      return ((bits >> np.uint64(bit)) & np.uint64(1)) == 1
71
72
73  def check_file(path, max_events):
74      t0 = time.time()
75      f = uproot.open(path)
76      t = f["Events"]
77      keys = set(t.keys())
78      title = t["TrigObj_filterBits"].title
79      dtype = str(t["TrigObj_filterBits"].interpretation)
80      docs = parse_bitdoc(title)
81      pho, ele = docs.get("Photon", {}), docs.get("Electron", {})
82      bits_unseeded = {k: find_bit(pho, v) for k, v in UNSEEDED.items()}
83      bits_seeded = {k: find_bit(pho, v) for k, v in SEEDED.items()}
84      bit_ht = find_bit(pho, HT_BIT)
85      bit_ele_unseeded = find_bit(ele, ELE_UNSEEDED_DOC)
86      paths = [p for p in PATHS if p in keys]
87      branches = ["TrigObj_id", "TrigObj_filterBits"] + paths
88      res = {
89          "file": path, "events": 0, "filterBits_dtype": dtype, "n_photon_bits_documented": len(pho),
90          "bits_unseeded": bits_unseeded, "bits_seeded": bits_seeded, "bit_hltHtEcal800": bit_ht,
```

Backup code: verify_nanoaod_files.py (4/6)

scripts/verify_nanoaod_files.py, lines 91 to 119

```
91          "bit_electron_caloIdL_MW_unseeded": bit_ele_unseeded,
92          "paths_present": paths, "fired": {p: 0 for p in paths},
93          "objects_per_id": {}, "photon_objects_with_bit": {}, "events_fired_mass90_with_photon_bit": {},
94          "electron_objects_with_unseeded_bit": 0,
95      }
96      allbits = {**{"unseeded:" + k: v for k, v in bits_unseeded.items()}, **{"seeded:" + k: v for k, v in bits_seeded.items()}, "hltHtEcal800":
            bit_ht}
97      for name in allbits:
98          res["photon_objects_with_bit"][name] = 0
99          res["events_fired_mass90_with_photon_bit"][name] = 0
100     main = "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90"
101     nread = 0
102     for arrays in t.iterate(branches, step_size="100 MB", entry_stop=max_events):
103         n = len(arrays["TrigObj_id"])
104         nread += n
105         ids = arrays["TrigObj_id"]
106         bits = ak.values_astype(arrays["TrigObj_filterBits"], np.uint64)
107         flat_ids = ak.to_numpy(ak.flatten(ids))
108         u, c = np.unique(flat_ids, return_counts=True)
109         for i, k in zip(u.tolist(), c.tolist()):
110             res["objects_per_id"][str(i)] = res["objects_per_id"].get(str(i), 0) + k
111         for p in paths:
112             res["fired"][p] += int(ak.sum(arrays[p]))
113         isph = ids == 22
114         fired_main = arrays[main] if main in keys else ak.zeros_like(arrays["TrigObj_id"][:, 0:0], dtype=bool)
115         for name, bit in allbits.items():
116             if bit is None:
117                 continue
118             sel = isph & has_bit(bits, bit)
119             res["photon_objects_with_bit"][name] += int(ak.sum(sel))
```

Backup code: verify_nanoaod_files.py (5/6)

scripts/verify_nanoaod_files.py, lines 120 to 149

```
120             if main in keys:
121                 res["events_fired_mass90_with_photon_bit"][name] += int(ak.sum(ak.any(sel, axis=1) & arrays[main]))
122         if bit_ele_unseeded is not None:
123             res["electron_objects_with_unseeded_bit"] += int(ak.sum((ids == 11) & has_bit(bits, bit_ele_unseeded)))
124     res["events"] = nread
125     res["seconds"] = round(time.time() - t0, 1)
126     # verdict
127     uns = [v for k, v in res["photon_objects_with_bit"].items() if k.startswith("unseeded:") and allbits[k] is not None]
128     sed = [v for k, v in res["photon_objects_with_bit"].items() if k.startswith("seeded:") and allbits[k] is not None]
129     if not any(b is not None for b in bits_unseeded.values()):
130         res["verdict"] = "diphoton unseeded bits not documented in this file (NanoAOD v11 or older): not affected"
131     elif sum(sed) > 0 and sum(uns) == 0:
132         res["verdict"] = "BUG REPRODUCED: seeded last-filter bits set %d times, unseeded last-filter bits never set" % sum(sed)
133     elif sum(uns) > 0:
134         res["verdict"] = "UNEXPECTED: unseeded bits set %d times (fixed file, or claim wrong: report this)" % sum(uns)
135     else:
136         res["verdict"] = "INCONCLUSIVE: no object with the seeded diphoton bits either (too few diphoton-triggered events?)"
137     return res
138
139
140 def main():
141     ap = argparse.ArgumentParser()
142     ap.add_argument("--label", required=True, help="NanoAOD version label, e.g. v13")
143     ap.add_argument("--files", nargs="+", required=True)
144     ap.add_argument("--max-events", type=int, default=300000)
145     ap.add_argument("--out", default=None)
146     a = ap.parse_args()
147     results = []
148     for p in a.files:
149         print("==", a.label, p, flush=True)
```

Backup code: verify_nanoaod_files.py (6/6)

scripts/verify_nanoaod_files.py, lines 150 to 163

```
150         try:
151             r = check_file(p, a.max_events)
152         except Exception as e:  # keep going, record the failure
153             r = {"file": p, "error": repr(e)}
154         results.append(r)
155         print(json.dumps({k: v for k, v in r.items() if k not in ("file",)}, indent=1), flush=True)
156     out = a.out or "results_%s.json" % a.label
157     json.dump({"label": a.label, "results": results}, open(out, "w"), indent=1)
158     print("wrote", out)
159
160
161 if __name__ == "__main__":
162     main()
```

Backup code: deadbits.py (1/9)

scripts/deadbits.py, lines 1 to 29

```
"""Resolve every trigger-object filter bit of a NanoAOD triggerObjects_cff.py against HLT menus.
usage: deadbits.py <triggerObjects_cff.py> <label=menu.py> [<label=menu.py> ...]
Status per bit and menu: OK (a matching filter whose objects the selection's coll() accepts),
COLL (matching filters exist but none passes the selection's coll()), STALE (no module matches),
UNK (matching filter, collection could not be determined)."""
import re, sys, ast

# ---------- menu ----------
def parse_modules(text):
    mods = {}
    for m in re.finditer(r'^(?:fragment|process)\.(\w+) = cms\.(\w+)\(', text, re.M):
        k = m.end(); depth = 1; q = None
        while k < len(text) and depth:
            c = text[k]
            if q:
                if c == q: q = None
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
        mods[m.group(1)] = (m.group(2), text[m.end():k-1])
    return mods

CAND_PARAMS = ["l1EGCand", "inputTag", "CandTag", "inputCandCollection", "candIsolatedTag",
               "candNonIsolatedTag", "inputTag1", "inputTag2", "firstLegLastFilter", "secondLegLastFilter",
               "JetSrc", "src", "htLabels", "mhtLabels", "inputJetTag", "originalTag", "TauSrc", "jetSrc",
               "Jets", "JetTags", "inputJets", "candTag", "PreviousCandTag", "previousCandTag", "metLabels",
               "mhtLabel", "inputMet", "L1TauTrigger", "TriggerSrc", "JetTag", "MuonTag", "ElectronTag",
               "PhotonTag", "muonTag", "electronTag", "photonTag", "tauTag", "jetTag", "taus", "originTag1", "originTag2", "originTag3", "
        originTag", "jets"]
```

Backup code: deadbits.py (2/9)

scripts/deadbits.py, lines 30 to 58

```
30
31  def tag_labels(body, name):
32      m = re.search(r'\b%s = cms\.(VInputTag|InputTag)\(\s*(.*?)\s*\)' % re.escape(name), body, re.S)
33      if not m: return None
34      kind, inner = m.group(1), m.group(2)
35      toks = [t.strip().strip('"\'') for t in inner.replace("\n", " ").split(",") if t.strip()]
36      if kind == "InputTag":
37          return [toks[0].split(":")[0]] if toks else []
38      return [t.split(":")[0] for t in toks]
39
40  def collections_of(mods, name, depth=0, seen=None):
41      """Collections (EDProducer labels) whose objects a filter saves."""
42      if seen is None: seen = set()
43      if name in seen or depth > 8 or name not in mods: return set()
44      seen.add(name)
45      typ, body = mods[name]
46      direct, follow = set(), []
47      for p in CAND_PARAMS:
48          labs = tag_labels(body, p)
49          if not labs: continue
50          for lab in labs:
51              t2 = mods.get(lab, ("?", ""))[0]
52              if t2 == "EDProducer": direct.add(lab)
53              elif t2 == "EDFilter": follow.append(lab)
54      if direct: return direct
55      out = set()
56      for f in follow: out |= collections_of(mods, f, depth + 1, seen)
57      return out
```

Backup code: deadbits.py (3/9)

scripts/deadbits.py, lines 59 to 88

```
59  def glob_re(pat):
60      parts = pat.split("*")
61      return re.compile("^" + ".*".join(re.escape(p) for p in parts) + "$")
62
63  # ---------- config ----------
64  def block(text, name):
65      lines = text.splitlines(); start = None
66      for i, l in enumerate(lines):
67          if re.match(r'^\s*%s\s*=\s*cms\.PSet\(' % name, l): start = i; break
68      if start is None: return None
69      indent = len(lines[start]) - len(lines[start].lstrip()); out = [lines[start]]
70      for l in lines[start+1:]:
71          ind = len(l) - len(l.lstrip())
72          if l.strip() and ind <= indent: break
73          out.append(l)
74      return "\n".join(out)
75
76  def split_args(s):
77      args, depth, q, cur = [], 0, None, ""
78      for c in s:
79          if q:
80              cur += c
81              if c == q: q = None
82              continue
83          if c in "\"'": q = c; cur += c; continue
84          if c in "([{": depth += 1
85          if c in ")]}": depth -= 1
86          if c == "," and depth == 0: args.append(cur); cur = ""; continue
87          cur += c
88      if cur.strip(): args.append(cur)
```

Backup code: deadbits.py (4/9)

scripts/deadbits.py, lines 89 to 112

```
    return [a.strip() for a in args]

def calls(blk, fname="mksel("):
    res, i = [], 0
    while True:
        j = blk.find(fname, i)
        if j < 0: break
        k = j + len(fname); depth = 1; q = None
        while k < len(blk) and depth:
            c = blk[k]
            if q:
                if c == q: q = None
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
        res.append(blk[j+len(fname):k-1]); i = k
    return res

def chaintoken(tokens, op):
    if not isinstance(tokens, (list, tuple)):
        tokens = [t.strip() for t in tokens.split(op)]
    return tokens
```

Backup code: deadbits.py (5/9)

scripts/deadbits.py, lines 113 to 141

```
113 def alternatives(sel0):
114     """Emulate mksel: return list of alternatives; each alternative = list of (kind, pattern) all required."""
115     s = sel0
116     if isinstance(s, str) and "OR" in s: s = chaintoken(s, "OR")
117     if isinstance(s, str) and "AND" in s: s = ("AND",) + tuple(chaintoken(s, "AND"))
118     if isinstance(s, tuple) and s and s[0] == "AND": return [[("filter", t) for t in s[1:]]]
119     if isinstance(s, list): return [[("filter", t)] for t in s]
120     if isinstance(s, tuple): return [[("filter", t) for t in s]]
121     toks = re.findall(r"(filter|path|coll)\('([^']*)'\)", s)
122     if not toks: return [[("raw", s)]]
123     if "min(" in s or "&&" in s:   # treat as AND of all tokens (approximation)
124         return [toks]
125     return [[t] for t in toks]
126 
127 def parse_config(text):
128     sels = {}
129     for name in ("Electron", "Photon", "Muon", "Tau", "BoostedTau", "Jet", "FatJet", "MET", "HT", "MHT"):
130         blk = block(text, name)
131         if blk is None: continue
132         sel = re.search(r'sel\s*=\s*cms\.string\(\(["\']\)(.*?)\1\)', blk)
133         skip = re.search(r'skipObjectsNotPassingQualityBits\s*=\s*cms\.bool\(\(\w+\)\)', blk)
134         colls = re.findall(r"coll\('([^']*)'\)", sel.group(2)) if sel else []
135         if sel and re.search(r"coll\('[^']*'\)\s*\|\|\s*filter\(", sel.group(2)): colls = []  # coll OR-ed with filters: not restrictive
136         bits = []
137         qb = blk[blk.find("qualityBits"):] if "qualityBits" in blk else ""
138         if "cms.VPSet" in qb:
139             for n, c in enumerate(calls(qb)):
140                 args = split_args(c)
141                 try: a0 = ast.literal_eval(args[0])
```

Backup code: deadbits.py (6/9)

scripts/deadbits.py, lines 142 to 165

```
                except Exception: a0 = args[0].strip('"')
                doc = ""
                if len(args) > 1 and not args[1].startswith("bit"):
                    try: doc = ast.literal_eval(args[1])
                    except Exception: doc = args[1]
                b = re.search(r'bit\s*=\s*(\d+)', c)
                bits.append((int(b.group(1)) if b else n, a0, doc))
        else:  # old string layout: "a*filter('x') + ..."
            m = re.search(r'qualityBits\s*=\s*cms\.string\(\s*(.*?)\)\s*,\s*\n', blk, re.S)
            if m:
                expr = "".join(ast.literal_eval(p) for p in re.findall(r'"(?:[^"\\]|\\.)*"', m.group(1)))
                for term in expr.split("+"):
                    w = re.match(r'\s*(\d+)\*', term)
                    bitno = int(w.group(1)).bit_length() - 1 if w else 0
                    pats = re.findall(r"filter\('([^']*)'\)", term)
                    bits.append((bitno, pats if len(pats) > 1 else (pats[0] if pats else term.strip()), ""))
        sels[name] = dict(sel=sel.group(2) if sel else "", skip=skip.group(1) if skip else "?", colls=colls, bits=bits)
    return sels

# ---------- evaluation ----------
def accepts(colls, label):
    if not colls: return True
    return any(glob_re(c).match(label) for c in colls)
```

Backup code: deadbits.py (7/9)

scripts/deadbits.py, lines 166 to 192

```
166 def token_status(mods, filters, paths, producers, kind, pat, colls):
167     if kind == "path":
168         return "OK" if any(glob_re(pat).match(p) for p in paths) else "STALE"
169     if kind == "coll":
170         return "OK" if any(glob_re(pat).match(p) for p in producers) else "STALE"
171     if kind == "raw":
172         return "UNK"
173     rx = glob_re(pat)
174     matched = [f for f in filters if rx.match(f)]
175     if not matched: return "STALE"
176     best, info = "COLL", []
177     for f in matched:
178         cs = collections_of(mods, f)
179         if not cs: best = "UNK" if best != "OK" else best; info.append("%s[%s:?]" % (f, mods[f][0])); continue
180         if any(accepts(colls, c) for c in cs): return "OK"
181         info.append("%s[%s:%s]" % (f, mods[f][0], ",".join(sorted(cs))))
182     return best + "{" + "; ".join(info[:3]) + "}"
183
184 RANK = {"OK": 0, "UNK": 1, "COLL": 2, "STALE": 3}
185 def bit_status(mods, filters, paths, producers, alts, colls):
186     results = []
187     for alt in alts:
188         sts = [token_status(mods, filters, paths, producers, k, p, colls) for k, p in alt]
189         worst = max(sts, key=lambda s: RANK[s.split("{")[0]])
190         results.append(worst)
191     return min(results, key=lambda s: RANK[s.split("{")[0]])
```

Backup code: deadbits.py (8/9)

scripts/deadbits.py, lines 193 to 221

```
193 def main():
194     cfg = open(sys.argv[1]).read()
195     sels = parse_config(cfg)
196     menus = []
197     for arg in sys.argv[2:]:
198         label, path = arg.split("=", 1)
199         mods = parse_modules(open(path).read())
200         filters = [n for n, (t, b) in mods.items() if t == "EDFilter"]
201         paths = [n for n, (t, b) in mods.items() if t == "Path"]
202         producers = [n for n, (t, b) in mods.items() if t == "EDProducer"]
203         menus.append((label, mods, filters, paths, producers))
204     print("config:", sys.argv[1]); print("menus:", " ".join(l for l, *_ in menus))
205     for name, s in sels.items():
206         print("\n== %s (sel colls: %s; skip=%s; %d bits)" % (name, s["colls"] or "any", s["skip"], len(s["bits"])))
207         if not s["bits"] and s["skip"] == "True":
208             print("   !! no quality bits and skipObjectsNotPassingQualityBits=True: this object type is never written")
209         counts = {l: {"OK": 0, "UNK": 0, "COLL": 0, "STALE": 0} for l, *_ in menus}
210         for bitno, a0, doc in s["bits"]:
211             alts = alternatives(a0)
212             row = []
213             for label, mods, filters, paths, producers in menus:
214                 st = bit_status(mods, filters, paths, producers, alts, s["colls"])
215                 counts[label][st.split("{")[0]] += 1
216                 row.append((label, st))
217             if any(st.split("{")[0] != "OK" for l, st in row):
218                 desc = a0 if isinstance(a0, str) else "|".join(a0)
219                 print("  bit %2d %-70s %s" % (bitno, (desc[:68] + (".." if len(desc) > 68 else "")), " ".join("%s:%s" % (l, st.split("{")[0])
        for l, st in row)))
220                 extra = [st for l, st in row if "{" in st]
221                 if extra: print("          detail: " + extra[0][extra[0].find("{"):][:260])
```

Backup code: deadbits.py (9/9)

scripts/deadbits.py, lines 222 to 225

```
222         print("   summary: " + " | ".join("%s OK=%d COLL=%d STALE=%d UNK=%d" % (l, c["OK"], c["COLL"], c["STALE"], c["UNK"]) for l, c in counts.
            items()))
223
224 main()
```

Backup code: menuscan.py (1/3)

scripts/menuscan.py, lines 1 to 26

```python
import re, sys

def parse_modules(text):
    mods = {}
    pat = re.compile(r'^(?:fragment|process)\.(\w+) = cms\.(\w+)\(', re.M)
    for m in pat.finditer(text):
        k = m.end(); depth = 1; q = None
        while k < len(text) and depth:
            c = text[k]
            if q:
                if c == q: q = None
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
        mods[m.group(1)] = (m.group(2), text[m.end():k-1])
    return mods

def param(body, name):
    m = re.search(r'\b%s = cms\.\w+\(\s*(.*?)\s*\)' % re.escape(name), body, re.S)
    return m.group(1).replace('"', '').replace("'", "").replace("\n", " ") if m else None

def members(body):
    # sequence/path members in order
    return re.findall(r'(?:fragment|process)\.(\w+)', body)
```

Backup code: menuscan.py (2/3)

scripts/menuscan.py, lines 27 to 54

```
27 def describe(mods, name):
28     if name not in mods: return "%s: NOT DEFINED" % name
29     typ, body = mods[name]
30     edtype = re.match(r'\s*"(\w+)"', body)
31     edtype = edtype.group(1) if edtype else "?"
32     bits = []
33     for p in ("saveTags", "l1EGCand", "inputTag", "candTag", "triggerType", "varTag", "lessThan", "useEt", "thrRegularEB", "thrRegularEE", "
           thrOverEEB", "thrOverEEE", "thrOverE2EB", "thrOverE2EE", "ncandcut", "etcutEB", "etcutEE", "lowerMassCut", "upperMassCut", "minPt", "
           minMass", "maxMass"):
34         v = param(body, p)
35         if v is not None: bits.append("%s=%s" % (p, v))
36     return "%s [%s] %s" % (name, edtype, " ".join(bits))
37
38 def main(path, pathre, maxdepth=2):
39     text = open(path).read()
40     m = re.search(r'tableName = cms\.string\(\s*"([^"]+)"', text)
41     print("#" * 110); print(path, "menu:", m.group(1) if m else "?")
42     mods = parse_modules(text)
43     names = sorted(n for n, (t, b) in mods.items() if t == "Path" and re.search(pathre, n))
44     print("paths:", ", ".join(names))
45     for n in names:
46         if not re.search(r'Diphoton30_(22|18)', n): continue
47         print("-" * 100); print("PATH", n)
48         seen = set()
49         def walk(name, depth):
50             typ, body = mods.get(name, ("?", ""))
51             for mem in members(body):
52                 if mem in seen: continue
53                 seen.add(mem)
54                 t2 = mods.get(mem, ("?", ""))[0]
```

Backup code: menuscan.py (3/3)

scripts/menuscan.py, lines 55 to 66

```
55                 if t2 == "Sequence" and depth < maxdepth and not mem.startswith(("HLTBeginSequence", "HLTEndSequence", "HLTDoLocalHcal", "
       HLTDoFull", "HLTPFClustering", "HLTEgamma", "HLTL1", "HLTDoLocalPixel", "HLTRecoPixel", "HLTElePixel", "HLTTrackReco", "HLTPFTrack", "
       HLTPFRecoSequence", "HLTDo")):
56                     print("  " * depth + "SEQ " + mem)
57                     walk(mem, depth + 1)
58                 elif t2 == "EDFilter":
59                     print("  " * depth + describe(mods, mem))
60                 elif t2 == "Sequence":
61                     print("  " * depth + "seq " + mem + " (not expanded)")
62         walk(n, 1)
63
64 if __name__ == "__main__":
65     main(sys.argv[1], sys.argv[2] if len(sys.argv) > 2 else r'HLT_(Diphoton|DoublePhoton)')
```

Backup code: menusummary.py (1/3)

scripts/menusummary.py, lines 1 to 30

```
import re, sys, hashlib

def parse_modules(text):
    mods = {}
    pat = re.compile(r'^(?:fragment|process)\.(\w+) = cms\.(\w+)\(', re.M)
    for m in pat.finditer(text):
        k = m.end(); depth = 1; q = None
        while k < len(text) and depth:
            c = text[k]
            if q:
                if c == q: q = None
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
        mods[m.group(1)] = (m.group(2), text[m.end():k-1])
    return mods

def param(body, name):
    m = re.search(r'\b%s = cms\.\w+\(\s*(.*?)\s*\)' % re.escape(name), body, re.S)
    return m.group(1).replace('"', '').replace("'", "").replace("\n", " ") if m else None

def members(body):
    return re.findall(r'(?:fragment|process)\.(\w+)', body)

KEY = ["hltDiEG22EtEta2p55UnseededFilter",
       "hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter",
       "hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter",
       "hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter",
       "hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter",
```

Backup code: menusummary.py (2/3)

scripts/menusummary.py, lines 31 to 60

```
31          "hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter",
32          "hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter",
33          "hltEG30Iso60b40eCaloId11b32eHE10b9eR9Id50b90eEcalIsoFilter",
34          "hltDiEG18TrackIso60Iso60b40eCaloId11b32eHE10b9eR9Id50b90eTrackIsoUnseededFilter",
35          "hltDiEG18EtUnseededFilter"]
36
37  def leaves(mods, name, seen=None):
38      if seen is None: seen = set()
39      out = []
40      typ, body = mods.get(name, ("?", ""))
41      for mem in members(body):
42          if mem in seen: continue
43          seen.add(mem)
44          t2 = mods.get(mem, ("?", ""))[0]
45          if t2 in ("Sequence", "Path"):
46              out += leaves(mods, mem, seen)
47          elif t2 == "EDFilter":
48              out.append(mem)
49      return out
50
51  for path in sys.argv[1:]:
52      text = open(path).read()
53      m = re.search(r'tableName = cms\.string\(\s*["\']([^"\']+)["\']', text)
54      mods = parse_modules(text)
55      print("#" * 100)
56      print("%s\n  menu: %s" % (path.split("/")[-1], m.group(1) if m else "?"))
57      names = sorted(n for n, (t, b) in mods.items() if t == "Path" and re.search(r'HLT_(Diphoton|DoublePhoton)', n))
58      print("  paths: " + ", ".join(n.replace("HLT_", "") for n in names))
59      for k in KEY:
60          if k not in mods:
```

Backup code: menusummary.py (3/3)

scripts/menusummary.py, lines 61 to 78

```
61              print("  KEY %-88s : not defined" % k); continue
62          typ, body = mods[k]
63          ed = re.match(r'\s*"(\w+)"', body).group(1)
64          print("  KEY %-88s : %s saveTags=%s l1EGCand=%s inputTag=%s triggerType=%s" % (k, ed, param(body, "saveTags"), param(body, "l1EGCand"),
            param(body, "inputTag"), param(body, "triggerType")))
65      for n in names:
66          if not re.search(r'Diphoton30_(22|18)', n): continue
67          lv = [f for f in leaves(mods, n) if f.startswith("hlt") and not f.startswith("hltPre") and mods[f][1].find("HLTL1TSeed") < 0 and not re.
            search(r'(Trigger|Bool|HLTPrescaler)', mods[f][1][:40])]
68          egf = [f for f in lv if re.search(r'EG|Diphoton|DiEG|DiPhoton|Mass|Photon', f)]
69          uns = [f for f in egf if "Unseeded" in f]
70          sed = [f for f in egf if "Unseeded" not in f and not re.search(r'Comb|Mass', f)]
71          comb = [f for f in egf if re.search(r'Comb|Mass', f)]
72          st = set(param(mods[f][1], "saveTags") for f in egf)
73          h = hashlib.md5(" ".join(egf).encode()).hexdigest()[:8]
74          print("  PATH %s: nEgFilters=%d chain=%s saveTags=%s" % (n.replace("HLT_", ""), len(egf), h, "/".join(sorted(x or "None" for x in st))))
75          print("       last seeded   : %s" % (sed[-1] if sed else "-"))
76          print("       last unseeded : %s" % (uns[-1] if uns else "-"))
77          print("       comb/mass     : %s" % ", ".join(comb))
```

Backup code: pathmatrix.py (1/1)

scripts/pathmatrix.py, lines 1 to 22

```
import re, sys
def paths(path):
    text = open(path).read()
    m = re.search(r'tableName = cms\.string\(\s*["\']([^"\']+)["\']', text)
    res = {}
    for pm in re.finditer(r'^(?:fragment|process)\.(HLT_(?:Diphoton|DoublePhoton|DiphotonMVA)\w*?)_v(\d+) = cms\.Path\(', text, re.M):
        res[pm.group(1)] = int(pm.group(2))
    return (m.group(1) if m else "?"), res
cols = []
allp = set()
for f in sys.argv[1:]:
    name, res = paths(f)
    cols.append((f.split("/")[-1].replace("menu_", "").replace(".py", ""), name, res))
    allp |= set(res)
print("columns:")
for i, (f, n, r) in enumerate(cols):
    print("  c%-2d %-32s %s" % (i, f, n))
print()
print("%-78s %s" % ("path", " ".join("c%-2d" % i for i in range(len(cols)))))
for p in sorted(allp):
    print("%-78s %s" % (p.replace("HLT_", ""), " ".join("%-3s" % (r.get(p, "-")) for f, n, r in cols)))
```

Backup code: bits.py (1/3)

scripts/bits.py, lines 1 to 30

```
import re, sys

def block(text, name):
    lines = text.splitlines()
    start = None
    for i, l in enumerate(lines):
        if re.match(r'^\s*%s\s*=\s*cms\.PSet\(' % name, l):
            start = i; break
    if start is None:
        return None
    indent = len(lines[start]) - len(lines[start].lstrip())
    out = [lines[start]]
    for l in lines[start+1:]:
        ind = len(l) - len(l.lstrip())
        if l.strip() and ind <= indent:
            break
        out.append(l)
    return "\n".join(out)

def calls(blk, fname="mksel("):
    res = []
    i = 0
    while True:
        j = blk.find(fname, i)
        if j < 0: break
        k = j + len(fname); depth = 1; q = None
        while k < len(blk) and depth:
            c = blk[k]
            if q:
                if c == q: q = None
```

Backup code: bits.py (2/3)

scripts/bits.py, lines 31 to 60

```
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
        res.append(blk[j+len(fname):k-1])
        i = k
    return res

for path in sys.argv[1:]:
    text = open(path).read()
    print("=" * 100); print(path)
    for name in ("Photon", "Electron"):
        blk = block(text, name)
        if blk is None:
            print("  %s: block not found" % name); continue
        sel = re.search(r'sel\s*=\s*cms\.string\(\(["\']\)(.*?)\1\)', blk)
        print("  %s sel: %s" % (name, sel.group(2) if sel else "?"))
        skip = re.search(r'skipObjectsNotPassingQualityBits\s*=\s*cms\.bool\((\w+)\)', blk)
        print("  %s skip: %s" % (name, skip.group(1) if skip else "?"))
        if name == "Electron":
            continue
        cs = calls(blk)
        if not cs:
            qb = re.search(r'qualityBits\s*=\s*cms\.string\(\(["\']\)(.*?)\1', blk, re.S)
            print("  old layout qualityBits string:", qb.group(2)[:2000] if qb else "none")
            continue
        for n, c in enumerate(cs):
            b = re.search(r'bit\s*=\s*(\d+)', c)
            flag = ""
            if "Unseeded" in c: flag = "  <-- unseeded"
```

Backup code: bits.py (3/3)

scripts/bits.py, lines 61 to 62

```
61                print("   %2d%s %s%s" % (n, (" (bit=%s)" % b.group(1)) if b else "", c.strip()[:150], flag))
```

Backup code: bitfilters.py (1/2)

scripts/bitfilters.py, lines 1 to 24

```
 1  import re, sys
 2  def parse_modules(text):
 3      mods = {}
 4      for m in re.finditer(r'^(?:fragment|process)\.(\w+) = cms\.(\w+)\(', text, re.M):
 5          k = m.end(); depth = 1; q = None
 6          while k < len(text) and depth:
 7              c = text[k]
 8              if q:
 9                  if c == q: q = None
10              elif c in "\"'": q = c
11              elif c == "(": depth += 1
12              elif c == ")": depth -= 1
13              k += 1
14          mods[m.group(1)] = (m.group(2), text[m.end():k-1])
15      return mods
16  def param(body, name):
17      m = re.search(r'\b%s = cms\.\w+\(\s*(.*?)\s*\)' % re.escape(name), body, re.S)
18      return m.group(1).replace('"', '').replace("'", "").strip() if m else None
19  FILTERS = ["hltEG33L1EG26HEFilter","hltEG50HEFilter","hltEG75HEFilter","hltEG90HEFilter","hltEG120HEFilter","hltEG150HEFilter","hltEG175HEFilter
          ","hltEG200HEFilter","hltHtEcal800","hltEG45EBTightIDTightIsoTrackIsoFilter","hltEG50EBTightIDTightIsoTrackIsoFilter","
          hltEG110EBTightIDTightIsoTrackIsoFilter","hltEG120EBTightIDTightIsoTrackIsoFilter","hltMu17Photon30IsoCaloIdPhotonlegTrackIsoFilter","
          hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter","hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter","
          hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter","hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter","
          hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter","
          hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter","hltEG22L1VBFLooseIsoEGHEFilter"]
20  menus = []
21  for f in sys.argv[1:]:
22      text = open(f).read()
23      mods = parse_modules(text)
24      row = {}
```

Backup code: bitfilters.py (2/2)

scripts/bitfilters.py, lines 25 to 36

```
25      for name in FILTERS:
26          if name not in mods: row[name] = "-"; continue
27          typ, body = mods[name]
28          coll = param(body, "l1EGCand") or param(body, "inputTag") or "?"
29          if "," in coll: coll = coll.split(",")[0]
30          row[name] = {"hltEgammaCandidates": "S", "hltEgammaCandidatesUnseeded": "U"}.get(coll, coll)
31      menus.append((f.split("/")[-1].replace("menu_","").replace(".py",""), row))
32  print("%-82s %s" % ("filter", " ".join("%-3s" % ("c%d" % i) for i in range(len(menus)))))
33  for i, (n, r) in enumerate(menus): print("  c%-2d %s" % (i, n))
34  for name in FILTERS:
35      print("%-82s %s" % (name, " ".join("%-3s" % r[name] for n, r in menus)))
```

Backup code: wfn.py (1/1)

scripts/wfn.py, lines 1 to 27

```
import re, sys
class WFN:
    def __init__(self, offset):
        self.offset = offset; self.index = 1; self.thousands = 0; self.hundreds = 0
    def __call__(self):
        v = float(f"{self.offset}.{self.thousands}{self.hundreds}{self.index:02d}"); self.index += 1; return v
    def subnext(self):
        self.hundreds += 1; self.index = 1
    def next(self, index=None):
        if index is None: self.thousands += 1; self.hundreds = 0
        else: self.thousands = index; self.hundreds = 0
        self.index = 1
src = open(sys.argv[1]).read()
m = re.search(r'_wfn\s*=\s*WFN\((\d+)\)', src)
w = WFN(int(m.group(1))) if m else None
print("counter init:", m.group(0) if m else "none (old scheme?)")
for i, line in enumerate(src.splitlines(), 1):
    s = line.strip()
    if s.startswith("#"): continue
    if re.match(r'_wfn\.next\(\s*\)', s): w.next()
    elif re.match(r'_wfn\.next\((\d+)\)', s): w.next(int(re.match(r'_wfn\.next\((\d+)\)', s).group(1)))
    elif re.match(r'_wfn\.subnext\(\)', s): w.subnext()
    elif "workflows[_wfn()]" in s:
        print("%-10s %s" % (w(), s.split("=", 1)[1].strip()[:110]))
    elif re.match(r'workflows\[([\d.]+)\]', s):
        print("%-10s %s" % (re.match(r'workflows\[([\d.]+)\]', s).group(1), s.split("=", 1)[1].strip()[:110]))
```

Backup code: measure_trigobj.py (1/8)

scripts/measure_trigobj.py, lines 1 to 30

```python
#!/usr/bin/env python3
"""Count TrigObj filter bits of the Diphoton30_22 photon legs in one NanoAOD file.

Bit numbers are parsed from the TrigObj_filterBits branch documentation, never hardcoded.
Usage: python3 measure_trigobj.py <version> <dataset> <file> <out.json> [<reference file>]
With a reference file, events are matched by (run, lumi, event) and their TrigObj content compared.
"""
import json
import re
import sys

import awkward as ak
import numpy as np
import uproot

MAX_EVENTS = 300000
PATH_MASS90 = "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90"
PATH_DIELE = "HLT_DoubleEle33_CaloIdL_MW"
PHOTON_FILTERS = {
    "R9UnseededLast": "hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter",
    "TrackIsoUnseededLast": "hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter",
    "EG18Unseeded": "hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter",
    "seeded_R9IdLast": "hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter",
    "seeded_EcalIsoLast": "hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter",
    "hltHtEcal800": "hltHtEcal800",
}
SEEDED = ("seeded_R9IdLast", "seeded_EcalIsoLast")
UNSEEDED = ("R9UnseededLast", "TrackIsoUnseededLast", "EG18Unseeded")
EXPECTED_PHOTON_BIT_COUNTS = (0, 12, 18, 21)
BIT_ENTRY = re.compile(r"(\d+)\s*=>\s*(.*?)(?=,\s*\d+\s*=>|\s+for\s+\w+\s*$)")
```

Backup code: measure_trigobj.py (2/8)

scripts/measure_trigobj.py, lines 31 to 58

```
31  EVENT_COLUMNS = ["run", "luminosityBlock", "event", "TrigObj_id", "TrigObj_filterBits", "TrigObj_pt"]
32
33
34  def bits_of(doc, obj):
35      # Total: time O(len(doc)), space O(n_bits)
36      bits = {}
37      for chunk in doc.split("; "):                          # list scan O(len) | each per-object chunk
38          if not chunk.strip().endswith(" for " + obj):      # -                | keep only this object's chunk
39              continue
40          for match in BIT_ENTRY.finditer(chunk):            # regex O(len)     | each "N => description"
41              bits[int(match.group(1))] = match.group(2).strip()  # dict insert O(1) | bit -> description
42      return bits
43
44
45  def find_bit(bits, name):
46      # Total: time O(n_bits), space O(1)
47      for bit, description in bits.items():                  # dict scan O(n)   | first bit naming the filter
48          if name in description:                            # substring O(len) | description contains name
49              return bit
50      return None
51
52
53  def has_bit(filter_bits, bit):
54      # Total: time O(n_obj), space O(n_obj)
55      shifted = filter_bits >> np.uint64(bit)                # jagged op O(n)   | move bit to position 0
56      return (shifted & np.uint64(1)) == 1                   # jagged op O(n)   | per-object bit set?
```

Backup code: measure_trigobj.py (3/8)

scripts/measure_trigobj.py, lines 59 to 82

```
59 def count_objects(ids, filter_bits, object_id, bit, event_mask=None):
60     # Total: time O(n_obj), space O(n_obj)
61     if bit is None:
62         return None
63     selected = (ids == object_id) & has_bit(filter_bits, bit)  # jagged mask O(n) | objects of this id with bit
64     if event_mask is not None:
65         selected = selected & event_mask                       # broadcast O(n)   | only in chosen events
66     return int(ak.sum(selected))                               # reduction O(n)   | number of objects
67
68
69 def bit_counts_per_id(ids, filter_bits, n_bits, skip_id):
70     # Total: time O(n_obj * n_bits), space O(n_obj)
71     flat_ids = ak.to_numpy(ak.flatten(ids))                    # array O(n)       | all object ids
72     flat_bits = ak.to_numpy(ak.flatten(filter_bits))           # array O(n)       | all object bit words
73     counts = {}
74     for object_id in np.unique(flat_ids):                      # sort O(n log n)  | each object type
75         if object_id == skip_id:
76             continue
77         words = flat_bits[flat_ids == object_id]               # mask O(n)        | bit words of this type
78         per_bit = [int(((words >> np.uint64(b)) & np.uint64(1)).sum()) for b in range(n_bits)]
79         counts[str(int(object_id))] = {str(b): c for b, c in enumerate(per_bit) if c > 0}
80     return counts
```

Backup code: measure_trigobj.py (4/8)

scripts/measure_trigobj.py, lines 83 to 111

```
83  def event_contents(arrays, skip_id=None):
84      # Total: time O(n_obj log n_obj), space O(n_obj)
85      contents = {}
86      rows = zip(arrays["run"], arrays["luminosityBlock"], arrays["event"],
87                 ak.to_list(arrays["TrigObj_id"]), ak.to_list(arrays["TrigObj_filterBits"]),
88                 ak.to_list(arrays["TrigObj_pt"]))
89      for run, lumi, event, ids, words, pts in rows:              # loop O(n_events) | one event at a time
90          objects = [(i, int(w) & 0xFFFFFFFFFFFFFFFF, p) for i, w, p in zip(ids, words, pts) if i != skip_id]
91          contents[(int(run), int(lumi), int(event))] = sorted(objects)  # dict O(1)  | key -> sorted objects
92      return contents
93
94
95  def compare_with_reference(arrays, reference_url):
96      # Total: time O(n_obj log n_obj), space O(n_obj)
97      reference = uproot.open(reference_url)["Events"].arrays(EVENT_COLUMNS, entry_stop=MAX_EVENTS)
98      this_all, ref_all = event_contents(arrays), event_contents(reference)
99      this_other, ref_other = event_contents(arrays, skip_id=22), event_contents(reference, skip_id=22)
100     common = set(this_all) & set(ref_all)                       # set join O(n)    | events in both files
101     identical_all = sum(1 for k in common if this_all[k] == ref_all[k])
102     identical_other = sum(1 for k in common if this_other[k] == ref_other[k])
103     return {
104         "reference_file": reference_url,
105         "events_in_reference": len(ref_all),
106         "events_in_common": len(common),
107         "events_identical_all_TrigObj": identical_all,
108         "events_identical_TrigObj_without_id22": identical_other,
109     }
```

Backup code: measure_trigobj.py (5/8)

scripts/measure_trigobj.py, lines 112 to 141

```
112 def main():
113     # Total: time O(n_events * n_obj), space O(n_events * n_obj)
114     version, dataset, url, out_path = sys.argv[1:5]
115     reference_url = sys.argv[5] if len(sys.argv) > 5 else None
116     root_file = uproot.open(url)
117     tree = root_file["Events"]
118     branch = tree["TrigObj_filterBits"]
119     doc = branch.title
120     photon_bits = bits_of(doc, "Photon")                    # dict             | photon bit -> name
121     electron_bits = bits_of(doc, "Electron")                # dict             | electron bit -> name
122     bit = {key: find_bit(photon_bits, name) for key, name in PHOTON_FILTERS.items()}
123     electron_bit = find_bit(electron_bits, "CaloIdL_MW unseeded")
124     n_bits = 64 if "u8" in str(branch.interpretation) else 32
125 
126     paths = [p for p in (PATH_MASS90, PATH_DIELE) if p in tree.keys()]
127     arrays = tree.arrays(EVENT_COLUMNS + paths, entry_stop=MAX_EVENTS)  # columnar read | up to 300k events
128     ids = arrays["TrigObj_id"]
129     filter_bits = ak.values_astype(arrays["TrigObj_filterBits"], np.uint64)
130     n_events = len(ids)
131     assert n_events > 0, "no events read"
132 
133     fired = arrays[PATH_MASS90] if PATH_MASS90 in paths else None
134     n_fired = int(ak.sum(fired)) if fired is not None else None
135 
136     photons_with_bit = {key: count_objects(ids, filter_bits, 22, b) for key, b in bit.items()}
137     photons_with_bit_fired = {key: count_objects(ids, filter_bits, 22, b, fired) if fired is not None else None
138                               for key, b in bit.items()}
139 
140     unseeded_any = ak.zeros_like(ids, dtype=bool)           # jagged bool O(n) | photon has any unseeded bit
141     for key in UNSEEDED:
```

Backup code: measure_trigobj.py (6/8)

scripts/measure_trigobj.py, lines 142 to 171

```
        if bit[key] is not None:
            unseeded_any = unseeded_any | ((ids == 22) & has_bit(filter_bits, bit[key]))
    offending = ak.any(unseeded_any, axis=1)                    # reduction O(n)   | events with such a photon
    offending_events = [[int(r), int(l), int(e)] for r, l, e in zip(arrays["run"][offending][:50],
                        arrays["luminosityBlock"][offending][:50], arrays["event"][offending][:50])]

    unseeded_pt = ak.to_numpy(ak.flatten(arrays["TrigObj_pt"][unseeded_any]))  # array O(n) | pt of those objects
    photons_per_event = ak.sum(ids == 22, axis=1)               # reduction O(n)   | photon objects per event
    n_fired_with_unseeded = int(ak.sum(offending & fired)) if fired is not None else None

    flat_ids = ak.to_numpy(ak.flatten(ids))
    unique_ids, counts = np.unique(flat_ids, return_counts=True)  # sort O(n log n) | objects per id
    objects_per_id = {int(i): int(c) for i, c in zip(unique_ids, counts)}

    result = {
        "version": version,
        "dataset": dataset,
        "file": url,
        "file_size_bytes": getattr(root_file.file.source, "num_bytes", None),
        "events_in_file": int(tree.num_entries),
        "events_read": n_events,
        "filterBits_interpretation": str(branch.interpretation),
        "n_photon_bits_documented": len(photon_bits),
        "n_electron_bits_documented": len(electron_bits),
        "photon_bit_numbers": bit,
        "electron_CaloIdL_MW_unseeded_bit": electron_bit,
        "paths_present": paths,
        "fired_Mass90": n_fired,
        "fired_DoubleEle33_CaloIdL_MW": int(ak.sum(arrays[PATH_DIELE])) if PATH_DIELE in paths else None,
        "photons_with_bit": photons_with_bit,
```

Backup code: measure_trigobj.py (7/8)

scripts/measure_trigobj.py, lines 172 to 201

```
172             "photons_with_bit_in_Mass90_events": photons_with_bit_fired,
173             "electrons_with_CaloIdL_MW_unseeded_bit": count_objects(ids, filter_bits, 11, electron_bit),
174             "objects_per_id": objects_per_id,
175             "n_id2_MET": objects_per_id.get(2, 0),
176             "n_id1515_BoostedTau": objects_per_id.get(1515, 0),
177             "n_id22_Photon": objects_per_id.get(22, 0),
178             "events_with_unseeded_photon_bit": int(ak.sum(offending)),
179             "first_offending_run_lumi_event": offending_events,
180             "Mass90_events_with_unseeded_photon_bit": n_fired_with_unseeded,
181             "fraction_Mass90_events_with_unseeded_photon_bit":
182                 n_fired_with_unseeded / n_fired if n_fired else None,
183             "photons_per_Mass90_event_mean": float(ak.mean(photons_per_event[fired])) if n_fired else None,
184             "unseeded_photon_pt": {
185                 "n": int(len(unseeded_pt)),
186                 "min": float(unseeded_pt.min()) if len(unseeded_pt) else None,
187                 "median": float(np.median(unseeded_pt)) if len(unseeded_pt) else None,
188                 "n_below_22": int((unseeded_pt < 22).sum()),
189             },
190             "nTrigObj_mean": float(ak.mean(ak.num(ids))),
191             "bit_counts_per_id_without_22": bit_counts_per_id(ids, filter_bits, n_bits, skip_id=22),
192             "photon_bits_documented": {str(k): v for k, v in sorted(photon_bits.items())},
193         }
194         if reference_url:
195             result["reference_comparison"] = compare_with_reference(arrays, reference_url)
196 
197         if len(photon_bits) not in EXPECTED_PHOTON_BIT_COUNTS:
198             print("WARNING: unexpected number of documented photon bits:", len(photon_bits))
199         if result["events_with_unseeded_photon_bit"] > 0:
200             print("NOTE: photon objects WITH an unseeded bit found:", result["events_with_unseeded_photon_bit"], "events")
```

Backup code: measure_trigobj.py (8/8)

scripts/measure_trigobj.py, lines 202 to 210

```
202     with open(out_path, "w") as out:
203         json.dump(result, out, indent=1)
204     skip = ("photon_bits_documented", "bit_counts_per_id_without_22")
205     print(json.dumps({k: v for k, v in result.items() if k not in skip}, indent=1))
206
207
208 if __name__ == "__main__":
209     main()
```

Backup code: make_fixed_nano.sh (1/3)

scripts/make_fixed_nano.sh, lines 1 to 29

```
#!/bin/bash
# Reproduce NanoAOD from one MiniAOD file twice, without and with the photon TrigObj fix.
# Usage: bash make_fixed_nano.sh v13|v15
set -eo pipefail

export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
source /cvmfs/cms.cern.ch/cmsset_default.sh

WORK=/eos/user/s/smahata/trigobj_check/cmssw
# parent MiniAOD of the central v13 file 4f45fe87-a958-4c8f-b88f-c322fec857f3.root
INPUT=root://cms-xrd-global.cern.ch//store/mc/Run3Summer23MiniAODv4/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/MINIAODSIM/130
      X_mcRun3_2023_realistic_v15_ext1-v4/2820000/74338d47-0b38-4e0b-83a2-ad85c7f4e533.root
CFF=PhysicsTools/NanoAOD/python/triggerObjects_cff.py

case "$1" in
    v13)
        RELEASE=CMSSW_13_3_1_patch1
        ERA=Run3_2023
        GT=133X_mcRun3_2023_realistic_ForNanov13_v1
        ;;
    v15)
        RELEASE=CMSSW_15_0_15_patch4
        ERA=Run3_2023,run3_nanoAOD_pre142X
        GT=150X_mcRun3_2023_realistic_v1    # processing string of Run3Summer23NanoAODv15 in DAS
        ;;
    *)
        echo "usage: $0 v13|v15"; exit 1
        ;;
esac
OUT=$WORK/$1
```

Backup code: make_fixed_nano.sh (2/3)

scripts/make_fixed_nano.sh, lines 30 to 59

```
30 mkdir -p "$OUT"
31
32 run_nano() {
33     local label=$1
34     cmsDriver.py nano_$label --step NANO --mc --scenario pp --nThreads 4 \
35         --eventcontent NANOAODSIM --datatier NANOAODSIM \
36         --conditions $GT --era $ERA \
37         --filein $INPUT --fileout file:$OUT/nano_$label.root \
38         --python_filename $OUT/nano_${label}_cfg.py -n -1
39 }
40
41 echo "=== step 1: release area $RELEASE"
42 cd $WORK
43 if [ ! -d $RELEASE ]; then
44     scram project CMSSW $RELEASE
45 fi
46 cd $RELEASE/src
47 eval $(scramv1 runtime -sh)
48
49 echo "=== step 2: check out PhysicsTools/NanoAOD and build"
50 if [ ! -d PhysicsTools/NanoAOD ]; then
51     git cms-addpkg PhysicsTools/NanoAOD
52 fi
53 git checkout -- $CFF
54 scram b -j 8
55
56 echo "=== step 3: NANO without the fix"
57 grep -n "coll('hltEgammaCandidates')\"" $CFF
58 cd $OUT
59 run_nano nofix
```

Backup code: make_fixed_nano.sh (3/3)

scripts/make_fixed_nano.sh, lines 60 to 76

```
60 cd $CMSSW_BASE/src
61
62 echo "=== step 4: apply the one-line fix"
63 sed -i "s#type(92) && pt > 15 && coll('hltEgammaCandidates')\"#type(92) \&\& pt > 15 \&\& (coll('hltEgammaCandidates') || coll('
       hltEgammaCandidatesUnseeded'))\"#" $CFF
64 git diff $CFF
65 CHANGED=$(git diff --numstat | awk '{print $1, $2}')
66 if [ "$CHANGED" != "1 1" ]; then
67     echo "ERROR: expected exactly one changed line, got: $CHANGED"; exit 1
68 fi
69 scram b -j 8
70
71 echo "=== step 5: NANO with the fix"
72 cd $OUT
73 run_nano fix
74 ls -l $OUT/nano_nofix.root $OUT/nano_fix.root
75 echo "=== done $1"
```

Backup code: relval_fix_check.sh (1/4)

scripts/relval_fix_check.sh, lines 1 to 30

```
#!/bin/bash
# Relval check of the photon TrigObj fix in CMSSW_16_1_3: clean area vs area with commit aebf3eb64.
# Steps 1-2 of each workflow run once (clean area); steps 3-4 run in both areas on the same step-2 output.
# Usage: bash relval_fix_check.sh
set -eo pipefail

export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
source /cvmfs/cms.cern.ch/cmsset_default.sh

RELEASE=CMSSW_16_1_3
WORK=/eos/user/s/smahata/trigobj_check/relval_16_1_3
FIX_COMMIT=aebf3eb64b9c525e77a99405f8a97aa254b335e9
CFF=PhysicsTools/NanoAOD/python/triggerObjects_cff.py
mkdir -p $WORK

echo "=== step 1: clean area and full workflows"
cd $WORK
if [ ! -d clean ]; then
    scram project -n clean CMSSW $RELEASE
fi
cd clean/src
eval $(scramv1 runtime -sh)
mkdir -p $WORK/run_clean && cd $WORK/run_clean
# 12452.0 generates its own GEN-SIM: its RelVal GEN-SIM input is not found at T2_CH_CERN (--ibeos)
if ! ls 12452.0_*/step3_inNANOEDMAODSIM.root > /dev/null 2>&1; then
    rm -rf 12452.0_*
    runTheMatrix.py -w upgrade -l 12452.0 -j 4
fi
# 141.046 RAW is on tape only in DAS; the IB EOS cache (--ibeos) has the file
if ! ls 141.046_*/step3_inNANOEDMAOD.root > /dev/null 2>&1; then
```

Backup code: relval_fix_check.sh (2/4)

scripts/relval_fix_check.sh, lines 31 to 59

```
31      if ! ls -d 141.046_* > /dev/null 2>&1; then
32          runTheMatrix.py -l 141.046 -i all --ibeos -j 4 || true
33      fi
34      cd 141.046_*
35      # of the 321 RAW files DAS lists for run 369978 (tape only), only this one is in the IB EOS cache
36      echo "root://eoscms.cern.ch//eos/cms/store/user/cmsbuild/store/data/Run2023D/EGamma0/RAW/v1/000/369/978/00000/0068cd83-0af4-4efc-9155-
            b2521e558ebb.root" > step1_dasquery.log
37      grep -E "cmsDriver.py step[234] " cmdLog > steps234.sh
38      cat steps234.sh
39      bash steps234.sh
40      ls -l step3_inNANOEDMAOD.root
41      cd $WORK/run_clean
42  fi
43
44  echo "=== step 2: fix area with the cherry-picked commit"
45  cd $WORK
46  if [ ! -d fix ]; then
47      scram project -n fix CMSSW $RELEASE
48  fi
49  cd fix/src
50  eval $(scramv1 runtime -sh)
51  if [ ! -d PhysicsTools/NanoAOD ]; then
52      git cms-addpkg PhysicsTools/NanoAOD
53      git fetch https://github.com/santanumahata/cmssw.git nano-photon-trigobj-unseeded
54      git cherry-pick $FIX_COMMIT
55  fi
56  git diff $RELEASE --numstat
57  CHANGED=$(git diff $RELEASE --numstat | awk '{print $1, $2, $3}')
58  if [ "$CHANGED" != "1 1 $CFF" ]; then
59      echo "ERROR: expected exactly one changed line in $CFF, got: $CHANGED"; exit 1
```

Backup code: relval_fix_check.sh (3/4)

scripts/relval_fix_check.sh, lines 60 to 89

```
fi
git diff $RELEASE
scram b -j 8

echo "=== step 3: rerun steps 3 and 4 in the fix area on the clean step-2 output"
for CLEAN_DIR in $WORK/run_clean/12452.0_* $WORK/run_clean/141.046_*; do
    FIX_DIR=$WORK/run_fix/$(basename $CLEAN_DIR)
    mkdir -p $FIX_DIR && cd $FIX_DIR
    ln -sf $CLEAN_DIR/step2.root step2.root
    grep -E "cmsDriver.py step[34] " $CLEAN_DIR/cmdLog > steps34.sh
    cat steps34.sh
    if [ ! -s steps34.sh ]; then
        echo "ERROR: no step3/step4 command in $CLEAN_DIR/cmdLog"; exit 1
    fi
    bash steps34.sh
done

echo "=== step 4: flat NANO from the EDM NANO of both workflows (both areas)"
for AREA in clean fix; do
    cd $WORK/$AREA/src && eval $(scramv1 runtime -sh)
    cd $WORK/run_$AREA/12452.0_*
    cmsDriver.py flat --step NONE --mc --conditions auto:phase1_2023_realistic --era Run3_2023 \
        --filein file:step3_inNANOEDMAODSIM.root --fileout file:step3_flatNANOAODSIM.root \
        --eventcontent NANOAODSIM --datatier NANOAODSIM -n -1 > flat_nano.log 2>&1
    cd $WORK/run_$AREA/141.046_*
    cmsDriver.py flat --step NONE --data --conditions auto:run3_data_relval --era Run3_2023 \
        --filein file:step3_inNANOEDMAOD.root --fileout file:step3_flatNANOAOD.root \
        --eventcontent NANOAOD --datatier NANOAOD -n -1 > flat_nano.log 2>&1
done
```

Backup code: relval_fix_check.sh (4/4)

scripts/relval_fix_check.sh, lines 90 to 99

```
90 echo "=== step 5: log check"
91 for AREA in clean fix; do
92     for LOG in $WORK/run_$AREA/*/step3_*.log; do
93         echo "$LOG: exceptions $(grep -c "Exception" $LOG || true)," \
94              "triggerObjectTable lines $(grep -c "triggerObjectTable" $LOG || true)"
95         grep "triggerObjectTable" $LOG | head -5 || true
96     done
97 done
98 echo "=== done"
```

Backup code: compare_dqm.py (1/2)

scripts/compare_dqm.py, lines 1 to 23

```python
#!/usr/bin/env python3
"""List the histograms that differ between two harvested DQM files.

Usage: python3 compare_dqm.py <clean DQM_V0001_*.root> <fix DQM_V0001_*.root> <out.json>
"""
import json
import sys

import numpy as np
import uproot


def histograms(path):
    # Total: time O(n_hist * n_bins), space O(n_hist * n_bins)
    found = {}
    with uproot.open(path) as root_file:
        for name, classname in root_file.classnames().items():  # dict scan O(n)   | every stored object
            if not classname.startswith(("TH1", "TH2", "TH3", "TProfile")):
                continue
            found[name.split(";")[0]] = root_file[name].values(flow=True)  # array O(bins) | bin contents
    return found
```

Backup code: compare_dqm.py (2/2)

scripts/compare_dqm.py, lines 24 to 48

```
def main():
    # Total: time O(n_hist * n_bins), space O(n_hist * n_bins)
    clean_path, fix_path, out_path = sys.argv[1:4]
    clean, fix = histograms(clean_path), histograms(fix_path)
    only_clean = sorted(set(clean) - set(fix))                  # set diff O(n)    | histograms lost by the fix
    only_fix = sorted(set(fix) - set(clean))                    # set diff O(n)    | histograms added by the fix
    differing = sorted(name for name in set(clean) & set(fix)  # set join O(n)    | same name, different bins
                       if clean[name].shape != fix[name].shape or not np.array_equal(clean[name], fix[name]))
    result = {
        "clean": clean_path,
        "fix": fix_path,
        "n_histograms_clean": len(clean),
        "n_histograms_fix": len(fix),
        "only_in_clean": only_clean,
        "only_in_fix": only_fix,
        "differing": differing,
    }
    with open(out_path, "w") as out:
        json.dump(result, out, indent=1)
    print(json.dumps({k: v for k, v in result.items()}, indent=1)[:6000])


if __name__ == "__main__":
    main()
```

Backup code: commands as run: (1/15)

slides/code/commands_as_run.sh, lines 1 to 30

```
# Shell commands I ran by hand for the 5 November 2026 checks, copied verbatim from the session log.
# They ran from my laptop; the work happened on lxplus over ssh. Not a script to execute as a whole.

# ===== 1. DAS lookups

# --- Query DAS for NanoAOD datasets per version pattern
ssh -o BatchMode=yes lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  for p in \
    "/EGamma/Run2018*-UL2018_MiniAODv2_NanoAODv9*/NANOAOD" \
    "/EGamma/Run2022*-PromptNanoAODv10*/NANOAOD" \
    "/EGamma/Run2022*-PromptNanoAODv11*/NANOAOD" \
    "/EGamma/Run2022*-22Sep2023-v*/NANOAOD" \
    "/GluGluHtoGG*/Run3Summer22*NanoAODv12*/NANOAODSIM" \
    "/EGamma/Run2022*-16Dec2023-v*/NANOAOD" \
    "/GluGluHto*G*/Run3Summer23NanoAODv13*/NANOAODSIM" \
    "/EGamma*/Run2024*-PromptReco-v*/NANOAOD" \
    "/GluGluHto*G*/*NanoAODv14*/NANOAODSIM" \
    "/GluGluHto*G*/Run3Summer2*NanoAODv15*/NANOAODSIM" \
    "/EGamma*/Run202*NanoAODv15*/NANOAOD"
  do echo "== $p"; dasgoclient -query="dataset=$p" 2>&1 | head -8; done' 2>&1 | grep -v '^\*'

# --- Query DAS with wider v14/v15 patterns
ssh -o BatchMode=yes lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  for p in \
    "/GluGluH*G*/*NanoAODv14*/NANOAODSIM" \
    "/GluGluH*G*/Run3Summer24*/NANOAODSIM" \
    "/GluGluH*G*/*NanoAODv15*/NANOAODSIM" \
    "/EGamma*/Run2024*NanoAODv15*/NANOAOD"
```

Backup code: commands as run: 1. DAS lookups (2/15)

slides/code/commands_as_run.sh, lines 31 to 58

```
31    do echo "== $p"; dasgoclient -query="dataset=$p" 2>&1 | head -8; done' 2>&1 | grep -v '^\*'
32
33  # --- Get first file LFN for each chosen dataset
34  ssh -o BatchMode=yes lxplus.cern.ch '
35    export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
36    for d in \
37      /EGamma/Run2018D-UL2018_MiniAODv2_NanoAODv9-v3/NANOAOD \
38      /EGamma/Run2022D-PromptNanoAODv10_v2-v1/NANOAOD \
39      /EGamma/Run2022F-PromptNanoAODv11_v1-v2/NANOAOD \
40      /GluGluHtoGG_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer22NanoAODv12-130X_mcRun3_2022_realistic_v5-v2/NANOAODSIM \
41      /EGamma/Run2022D-22Sep2023-v1/NANOAOD \
42      /GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23NanoAODv13-133X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/NANOAODSIM \
43      /EGamma/Run2022D-16Dec2023-v1/NANOAOD \
44      /EGamma0/Run2024D-PromptReco-v1/NANOAOD \
45      /GluGluH-Hto2G_Par-M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/RunIII2024Summer24NanoAODv15-150X_mcRun3_2024_realistic_v2-v2/NANOAODSIM \
46      /EGamma/Run2022D-NanoAODv15-v1/NANOAOD
47    do echo "== $d"; dasgoclient -query="file dataset=$d" 2>&1 | head -1; done' 2>&1 | grep -v '^\*'
48
49  # --- Look up DAS datasets needed for tasks A, B, C
50  ssh -o BatchMode=yes lxplus.cern.ch '
51    export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
52    for p in \
53      "/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23MiniAODv4-130X_mcRun3_2023_realistic_v14-v2/MINIAODSIM" \
54      "/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23MiniAODv4*/MINIAODSIM" \
55      "/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23NanoAODv1*/NANOAODSIM" \
56      "/GluGluHto*G*/Run3Summer23NanoAODv15*/NANOAODSIM" \
57      "/EGamma0/Run2024*-MINIv6NANOv15-v*/NANOAOD" \
58      "/GluGluHtoGG*/*Winter24*NanoAOD*/NANOAODSIM"
```

Backup code: commands as run: 1. DAS lookups (3/15)

slides/code/commands_as_run.sh, lines 59 to 78

```
59    do echo "== $p"; dasgoclient -query="dataset=$p" 2>&1 | head -6; done' 2>&1 | grep -v '^\*'
60
61  # --- Check MiniAOD dataset status, central-file parent, wider patterns
62  ssh -o BatchMode=yes lxplus.cern.ch '
63    export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
64    for d in /GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23MiniAODv4-130X_mcRun3_2023_realistic_v14-v2/MINIAODSIM /
          GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23MiniAODv4-130X_mcRun3_2023_realistic_v15_ext1-v4/MINIAODSIM; do
65      echo "== $d"; dasgoclient -query="dataset dataset=$d status=*" -json 2>/dev/null | grep -o "\"dataset_access_type\":\"[A-Z]*\"" | head -1;
          dasgoclient -query="summary dataset=$d" 2>&1 | head -2; done
66    echo "== parent of central v13 file"
67    dasgoclient -query="parent file=/store/mc/Run3Summer23NanoAODv13/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/NANOAODSIM/133
          X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/2820000/4f45fe87-a958-4c8f-b88f-c322fec857f3.root"
68    for p in "/*/Run3Summer23NanoAODv15*/NANOAODSIM" "/GluGluH*/*Winter24*/NANOAODSIM" "/*/RunIII2024Winter24NanoAOD*/NANOAODSIM"; do echo "== $p"
          ; dasgoclient -query="dataset=$p" 2>&1 | head -4; done' 2>&1 | grep -v '^\*'
69
70  # --- Get first file of 2024D MINIv6NANOv15 and Winter24 datasets
71  ssh -o BatchMode=yes -o ConnectTimeout=30 lxplus.cern.ch '
72    export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
73    for d in /EGamma0/Run2024D-MINIv6NANOv15-v1/NANOAOD /GluGluHToGG_M-125_TuneCP5_13p6TeV_powheg-pythia8/Run3Winter24NanoAOD-133
          X_mcRun3_2024_realistic_v9-v2/NANOAODSIM; do
74      echo "== $d"; dasgoclient -query="file dataset=$d" | head -1; done' 2>&1 | grep -v '^\*'
75
76  # --- Check DAS sites and IB EOS cache for the RAW file
77  timeout 120 ssh -o BatchMode=yes -o ConnectTimeout=30 lxplus.cern.ch 'export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms; F=/
          store/data/Run2023D/EGamma0/RAW/v1/000/369/978/00000/0068cd83-0af4-4efc-9155-b2521e558ebb.root; echo "== DAS sites"; dasgoclient -query=
          "site file=$F"; echo "== ibeos cache"; ls -la /eos/cms/store/user/cmsbuild$F 2>&1 | head -2; ls /eos/cms/store/user/cmsbuild/store/data/
          Run2023D/EGamma0/RAW/v1/000/369/978/00000/ 2>&1 | head -3' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 1. DAS lookups (4/15)

slides/code/commands_as_run.sh, lines 79 to 100

```
# --- Check which 141.046 input files exist in IB EOS cache
timeout 120 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=30 smahata@lxplus967.cern.ch 'cd /eos/user/s/smahata/
       trigobj_check/relval_16_1_3/run_clean/141.046_*; wc -l < step1_dasquery.log; cat step1_lumiRanges.log | head -c 300; echo; for f in $(
       cat step1_dasquery.log); do if [ -f /eos/cms/store/user/cmsbuild$f ]; then echo "cached $f"; else echo "missing $f"; fi; done | sort |
       uniq -c -w 7 | head; ls /eos/cms/store/user/cmsbuild/store/data/Run2023D/EGamma0/RAW/v1/000/369/978/00000/ | wc -l' 2>&1 | grep -v '^\*'

# ===== 2. Measuring the NanoAOD files

# --- Copy script to lxplus and run v13 smoke test
cd <session folder> && scp -q -o BatchMode=yes measure_trigobj.py lxplus.cern.ch:/eos/user/s/smahata/trigobj_check/ 2>&1 | grep -v '^\*'; ssh -o
       BatchMode=yes lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check
  mkdir -p nanoaod-trigobj-notes/results
  python3 measure_trigobj.py v13 \
    /GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23NanoAODv13-133X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/NANOAODSIM \
    root://cms-xrd-global.cern.ch//store/mc/Run3Summer23NanoAODv13/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/NANOAODSIM/133
       X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/2820000/4f45fe87-a958-4c8f-b88f-c322fec857f3.root \
    nanoaod-trigobj-notes/results/v13_GluGluHto2G_M125_Run3Summer23.json' 2>&1 | grep -v '^\*'

# --- Run measurement on the remaining 9 NanoAOD files
ssh -o BatchMode=yes lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check
  R=root://cms-xrd-global.cern.ch/
```

Backup code: commands as run: 2. Measuring the NanoAOD files (5/15)

slides/code/commands_as_run.sh, lines 101 to 112

```
101  run() { echo "##### $1 $4"; python3 measure_trigobj.py "$1" "$2" "$R$3" "nanoaod-trigobj-notes/results/$4.json" 2>&1 | grep -E "WARNING|Error|
           error|\"(events_read|filterBits_interpretation|n_photon_bits_documented|fired_Mass90|fired_DoubleEle33_CaloIdL_MW|
           electron_CaloIdL_MW_unseeded_bit|electrons_with_CaloIdL_MW_unseeded_bit|n_id2_MET|n_id1515_BoostedTau|n_id22_Photon|
           events_with_unseeded_photon_bit)\""; }
102  run v9  /EGamma/Run2018D-UL2018_MiniAODv2_NanoAODv9-v3/NANOAOD /store/data/Run2018D/EGamma/NANOAOD/UL2018_MiniAODv2_NanoAODv9-v3/250000/566
           E12C7-CD3E-3F4D-A657-A71FF4CB2637.root v9_EGamma_Run2018D_UL2018NanoAODv9
103  run v10 /EGamma/Run2022D-PromptNanoAODv10_v2-v1/NANOAOD /store/data/Run2022D/EGamma/NANOAOD/PromptNanoAODv10_v2-v1/2520000/3f1fe826-1729-4ba9
           -83b9-d28a2cfd2b88.root v10_EGamma_Run2022D_PromptNanoAODv10
104  run v11 /EGamma/Run2022F-PromptNanoAODv11_v1-v2/NANOAOD /store/data/Run2022F/EGamma/NANOAOD/PromptNanoAODv11_v1-v2/80000/6d07f113-b587-45b3-8
           db6-c2928358ae0b.root v11_EGamma_Run2022F_PromptNanoAODv11
105  run v12 /GluGluHtoGG_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer22NanoAODv12-130X_mcRun3_2022_realistic_v5-v2/NANOAODSIM /store/mc/
           Run3Summer22NanoAODv12/GluGluHtoGG_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/NANOAODSIM/130X_mcRun3_2022_realistic_v5-v2/2530000/
           fd1a30c4-ac59-4c33-8245-719b82fad9f0.root v12_GluGluHtoGG_M125_Run3Summer22
106  run v12 /EGamma/Run2022D-22Sep2023-v1/NANOAOD /store/data/Run2022D/EGamma/NANOAOD/22Sep2023-v1/2530000/e5762711-8726-4523-8891-32efa9ddb381.
           root v12_EGamma_Run2022D_22Sep2023
107  run v13 /EGamma/Run2022D-16Dec2023-v1/NANOAOD /store/data/Run2022D/EGamma/NANOAOD/16Dec2023-v1/2550000/499b6a69-bd7e-4bce-bd6d-4ab4f250ea2d.
           root v13_EGamma_Run2022D_16Dec2023
108  run v14 /EGamma0/Run2024D-PromptReco-v1/NANOAOD /store/data/Run2024D/EGamma0/NANOAOD/PromptReco-v1/000/380/306/00000/9bfae790-c112-48bf-bc54-
           ef1feaa74a96.root v14_EGamma0_Run2024D_PromptReco
109  run v15 /GluGluH-Hto2G_Par-M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/RunIII2024Summer24NanoAODv15-150X_mcRun3_2024_realistic_v2-v2/NANOAODSIM
           /store/mc/RunIII2024Summer24NanoAODv15/GluGluH-Hto2G_Par-M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/NANOAODSIM/150
           X_mcRun3_2024_realistic_v2-v2/120000/acebfb52-a25b-48bc-b9f6-80fe54a98d56.root v15_GluGluHHto2G_M125_RunIII2024Summer24
110  run v15 /EGamma/Run2022D-NanoAODv15-v1/NANOAOD /store/data/Run2022D/EGamma/NANOAOD/NanoAODv15-v1/2820000/41628bc7-c70f-4bf7-b2a4-e755aea819fd.
           root v15_EGamma_Run2022D_NanoAODv15
111  echo ALLDONE' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 2. Measuring the NanoAOD files (6/15)

slides/code/commands_as_run.sh, lines 113 to 128

```
113 # --- Measure 2024D MINIv6NANOv15 and Winter24 files
114 ssh -o BatchMode=yes -o ServerAliveInterval=15 lxplus.cern.ch '
115   export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
116   source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
117   cd /eos/user/s/smahata/trigobj_check
118   R=root://cms-xrd-global.cern.ch/
119   K="events_read|events_in_file|filterBits_interpretation|n_photon_bits_documented|fired_Mass90|photons_with_bit|R9|TrackIso|EG18|seeded|
          hltHtEcal|electron_CaloIdL|electrons_with|n_id2|n_id1515|n_id22|events_with_unseeded|nTrigObj_mean|file_size|WARNING|NOTE|Error"
120   python3 measure_trigobj.py v15 /EGamma0/Run2024D-MINIv6NANOv15-v1/NANOAOD $R/store/data/Run2024D/EGamma0/NANOAOD/MINIv6NANOv15-v1/110000/9
          d4e97c7-f23a-4cff-ac72-58488e544076.root nanoaod-trigobj-notes/results/v15_EGamma0_Run2024D_MINIv6NANOv15.json 2>&1 | grep -E "$K" |
          head -40
121   echo "#####"
122   python3 measure_trigobj.py Winter24_133X /GluGluHToGG_M-125_TuneCP5_13p6TeV_powheg-pythia8/Run3Winter24NanoAOD-133X_mcRun3_2024_realistic_v9-
          v2/NANOAODSIM $R/store/mc/Run3Winter24NanoAOD/GluGluHToGG_M-125_TuneCP5_13p6TeV_powheg-pythia8/NANOAODSIM/133X_mcRun3_2024_realistic_v9-
          v2/80000/68618af9-2d38-4ef7-9aa9-2d0c30a6e3f1.root nanoaod-trigobj-notes/results/Winter24_133X_GluGluHToGG_M125_powheg.json 2>&1 | grep -
          E "$K" | head -40' 2>&1 | grep -v '^\*'
123
124 # ===== 3. Fix test, v13 and v15
125
126 # --- Copy scripts and start NANO runs on lxplus967 as smahata
127 cd <session folder> && O="-o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=15"; scp -q $O make_fixed_nano.sh
          measure_trigobj.py smahata@lxplus967.cern.ch:/eos/user/s/smahata/trigobj_check/ 2>&1 | grep -v '^\*'; ssh $O smahata@lxplus967.cern.ch '
          cd /eos/user/s/smahata/trigobj_check && nohup bash -c "bash make_fixed_nano.sh v13; bash make_fixed_nano.sh v15" > make_fixed_nano.log
          2>&1 < /dev/null & sleep 8; hostname; head -5 /eos/user/s/smahata/trigobj_check/make_fixed_nano.log' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 3. Fix test, v13 and v15 (7/15)

slides/code/commands_as_run.sh, lines 129 to 150

```
# --- Rerun v13 nano_fix cmsRun on lxplus967 in held-open session
ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  cd /eos/user/s/smahata/trigobj_check/cmssw/CMSSW_13_3_1_patch1/src && eval $(scramv1 runtime -sh)
  git diff --numstat
  cd /eos/user/s/smahata/trigobj_check/cmssw/v13
  echo "=== v13 nano_fix rerun start $(date)"
  cmsRun nano_fix_cfg.py > nano_fix_rerun.log 2>&1
  echo "=== v13 nano_fix rerun exit $? $(date)"
  ls -l nano_nofix.root nano_fix.root' 2>&1 | grep -v '^\*'

# --- Run v15 fix-test NANO pair on another lxplus node
ssh -o BatchMode=yes -o ServerAliveInterval=30 lxplus.cern.ch 'hostname; cd /eos/user/s/smahata/trigobj_check && bash make_fixed_nano.sh v15 >
       make_fixed_nano_v15.log 2>&1; echo "v15 exit $? $(date)"; grep -E "^=== |^ERROR:|^[0-9]+: +sel|^[-+] +sel" make_fixed_nano_v15.log' 2>&1
        | grep -v '^\*'

# --- Measure central v13 file into fix_test results
ssh -o BatchMode=yes -o ServerAliveInterval=30 lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check && mkdir -p nanoaod-trigobj-notes/results/fix_test
  python3 measure_trigobj.py v13_central /GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23NanoAODv13-133
       X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/NANOAODSIM root://cms-xrd-global.cern.ch//store/mc/Run3Summer23NanoAODv13/GluGluHto2G_M
       -125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/NANOAODSIM/133X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/2820000/4f45fe87-a958-4c8f-b88f-
       c322fec857f3.root nanoaod-trigobj-notes/results/fix_test/central_v13.json 2>&1 | grep -E "events_read|fired_Mass90|nTrigObj_mean|
       file_size|events_with_unseeded|fraction|photons_per|Error|WARN"' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 3. Fix test, v13 and v15 (8/15)

slides/code/commands_as_run.sh, lines 151 to 172

```
# --- Measure v13 nano_nofix and nano_fix with references
ssh -o BatchMode=yes -o ServerAliveInterval=30 lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check; V=cmssw/v13; O=nanoaod-trigobj-notes/results/fix_test
  CENTRAL=root://cms-xrd-global.cern.ch//store/mc/Run3Summer23NanoAODv13/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/NANOAODSIM/133
       X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/2820000/4f45fe87-a958-4c8f-b88f-c322fec857f3.root
  K="events_read|fired_Mass90|R9|TrackIso|EG18|seeded|hltHtEcal|nTrigObj_mean|file_size|events_with_unseeded|fraction|photons_per|n_id22|\"n\"|
       min|median|below|reference|events_in|identical|Error|WARN|NOTE"
  python3 measure_trigobj.py v13_nofix MiniAOD-parent-of-central-v13 $V/nano_nofix.root $O/nano_nofix.json $CENTRAL 2>&1 | grep -E "$K"
  echo "#####"
  python3 measure_trigobj.py v13_fix MiniAOD-parent-of-central-v13 $V/nano_fix.root $O/nano_fix.json $V/nano_nofix.root 2>&1 | grep -E "$K"
  python3 -c "import json; a=json.load(open(\"$O/nano_nofix.json\")); b=json.load(open(\"$O/nano_fix.json\")); print(\"objects_per_id nofix\", a
       [\"objects_per_id\"]); print(\"objects_per_id fix  \", b[\"objects_per_id\"]); print(\"id!=22 bit counts identical:\", a[\"
       bit_counts_per_id_without_22\"]==b[\"bit_counts_per_id_without_22\"])"' 2>&1 | grep -v '^\*'

# --- Measure v15 nano_nofix and nano_fix
ssh -o BatchMode=yes -o ServerAliveInterval=30 lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check; V=cmssw/v15; O=nanoaod-trigobj-notes/results/fix_test
  ls -l $V/*.root
  K="events_read|events_in_file|interpretation|n_photon_bits|fired_Mass90|R9|TrackIso|EG18|seeded|hltHtEcal|nTrigObj_mean|file_size|
       events_with_unseeded|fraction|photons_per|n_id22|\"n\"|min|median|below|events_in|identical|Error|WARN|NOTE"
  python3 measure_trigobj.py v15_nofix MiniAOD-parent-of-central-v13 $V/nano_nofix.root $O/v15_nano_nofix.json 2>&1 | grep -E "$K"
  echo "#####"
  python3 measure_trigobj.py v15_fix MiniAOD-parent-of-central-v13 $V/nano_fix.root $O/v15_nano_fix.json $V/nano_nofix.root 2>&1 | grep -E "$K"
```

Backup code: commands as run: 3. Fix test, v13 and v15 (9/15)

slides/code/commands_as_run.sh, lines 173 to 191

```
173   python3 -c "import json; a=json.load(open(\"$0/v15_nano_nofix.json\")); b=json.load(open(\"$0/v15_nano_fix.json\")); print(\"objects_per_id
          nofix\", a[\"objects_per_id\"]); print(\"objects_per_id fix  \", b[\"objects_per_id\"]); print(\"id!=22 bit counts identical:\", a[\"
          bit_counts_per_id_without_22\"]==b[\"bit_counts_per_id_without_22\"])"' 2>&1 | grep -v '^\*'
174
175 # ===== 4. Relval in CMSSW_16_1_3
176
177 # --- List the five relval workflows in CMSSW_16_1_3
178 ssh -o BatchMode=yes -o ConnectTimeout=30 -o ServerAliveInterval=15 lxplus.cern.ch 'source /cvmfs/cms.cern.ch/cmsset_default.sh
179   cd /cvmfs/cms.cern.ch/el9_amd64_gcc13/cms/cmssw/CMSSW_16_1_3/src && eval $(scramv1 runtime -sh)
180   mkdir -p /tmp/smahata_rtm && cd /tmp/smahata_rtm && runTheMatrix.py -n -e -l 12452.0,12852.0,12434.0,12834.0,141.046 2>&1 | grep -v "^$" | cut
          -c1-400 | head -60' 2>&1 | grep -v '^\*'
181
182 # --- List relval workflows with upgrade and default matrices
183 ssh -o BatchMode=yes -o ConnectTimeout=30 -o ServerAliveInterval=15 lxplus.cern.ch 'source /cvmfs/cms.cern.ch/cmsset_default.sh
184   cd /cvmfs/cms.cern.ch/el9_amd64_gcc13/cms/cmssw/CMSSW_16_1_3/src && eval $(scramv1 runtime -sh)
185   mkdir -p /tmp/smahata_rtm && cd /tmp/smahata_rtm
186   echo "##### upgrade 12452/12852"; runTheMatrix.py -n -e -w upgrade -l 12452.0,12852.0 2>&1 | grep -v "^processing\|^ignoring\|^$" | cut -c1
          -300 | head -30
187   echo "##### default 12434/12834/141.046"; runTheMatrix.py -n -e -l 12434.0,12834.0,141.046 2>&1 | grep -v "^processing\|^ignoring\|^$" | cut -
          c1-300 | head -40' 2>&1 | grep -v '^\*'
188
189 # --- Run relval check with cached RAW file for 141.046
190 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch 'cd /eos/user/s/smahata/trigobj_check &&
          nohup bash relval_fix_check.sh > relval_fix_check.log 2>&1 < /dev/null; echo "relval exit code $?"; grep -E "^=== |^ERROR:|tests passed
          |Step[0-9]-|^[0-9]+\s+[0-9]+\s+Phys|exceptions|Fatal" relval_fix_check.log | cut -c1-250' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 4. Relval in CMSSW_16_1_3 (10/15)

slides/code/commands_as_run.sh, lines 192 to 214

```
# --- Dry-run 141.046 to get full step3/step4 commands
timeout 300 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  cd /eos/user/s/smahata/trigobj_check/relval_16_1_3/clean/src && eval $(scramv1 runtime -sh)
  mkdir -p /tmp/smahata_dry && cd /tmp/smahata_dry && rm -rf 141.046_*
  runTheMatrix.py -l 141.046 -i all --ibeos --dryRun > dry.log 2>&1
  grep -E "cmsDriver.py step[34] " 141.046_*/cmdLog' 2>&1 | grep -v '^\*'

# --- Run clean 141.046 steps 3-4 then the rest of relval script
ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  cd /eos/user/s/smahata/trigobj_check/relval_16_1_3/clean/src && eval $(scramv1 runtime -sh)
  cd /eos/user/s/smahata/trigobj_check/relval_16_1_3/run_clean/141.046_*
  grep -E "cmsDriver.py step[34] " /tmp/smahata_dry/141.046_*/cmdLog > steps34_from_dryrun.sh
  echo "# steps 3-4 below taken from runTheMatrix --dryRun (aborted run had not written them)" >> cmdLog
  cat steps34_from_dryrun.sh >> cmdLog
  echo "=== clean 141.046 steps 3-4 start $(date)"
  bash steps34_from_dryrun.sh; echo "steps34 exit $?"
  ls -l step3_inNANOEDMAOD.root step3_inDQM.root DQM_V0001*.root
  cd /eos/user/s/smahata/trigobj_check
  bash relval_fix_check.sh > relval_fix_check.log 2>&1; echo "relval exit code $? $(date)"
  grep -E "^=== |^ERROR:|^[0-9]+\s+[0-9]+\s+Phys|exceptions" relval_fix_check.log | cut -c1-250' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 4. Relval in CMSSW_16_1_3 (11/15)

slides/code/commands_as_run.sh, lines 215 to 240

```
# --- Rerun 141.046 steps 3-4 in the fix area
ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  W=/eos/user/s/smahata/trigobj_check/relval_16_1_3
  cd $W/fix/src && eval $(scramv1 runtime -sh) && git diff CMSSW_16_1_3 --numstat
  cd $W/run_fix/141.046_RunEGamma2023D
  cp $W/run_clean/141.046_RunEGamma2023D/steps34_from_dryrun.sh .
  echo "=== fix 141.046 steps 3-4 start $(date)"
  bash steps34_from_dryrun.sh; echo "=== fix 141.046 steps 3-4 exit $? $(date)"
  ls -l step3_inNANOEDMAOD.root DQM_V0001*.root' 2>&1 | grep -v '^\*'

# --- Convert, measure and compare 12452.0 clean vs fix
timeout 900 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=60 -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  W=/eos/user/s/smahata/trigobj_check/relval_16_1_3; T=/eos/user/s/smahata/trigobj_check
  for AREA in clean fix; do
    (cd $W/$AREA/src && eval $(scramv1 runtime -sh) && cd $W/run_$AREA/12452.0_* && \
     cmsDriver.py flat --step NONE --mc --conditions auto:phase1_2023_realistic --era Run3_2023 \
       --filein file:step3_inNANOEDMAODSIM.root --fileout file:step3_flatNANOAODSIM.root \
       --eventcontent NANOAODSIM --datatier NANOAODSIM -n -1 > flat_nano.log 2>&1; echo "$AREA flat exit $?"; ls -l step3_flatNANOAODSIM.root)
  done
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  O=$T/nanoaod-trigobj-notes/results/relval_16_1_3; mkdir -p $O
  C=$(ls $W/run_clean/12452.0_*/step3_flatNANOAODSIM.root); F=$(ls $W/run_fix/12452.0_*/step3_flatNANOAODSIM.root)
  cd $T
  python3 measure_trigobj.py 16_1_3_clean wf12452.0 $C $O/wf12452_clean.json 2>&1 | grep -E "events_read|fired_Mass90|n_photon_bits|n_id22|
       events_with_unseeded|fraction|nTrigObj_mean|Error"
```

Backup code: commands as run: 4. Relval in CMSSW_16_1_3 (12/15)

slides/code/commands_as_run.sh, lines 241 to 255

```
241   python3 measure_trigobj.py 16_1_3_fix wf12452.0 $F $O/wf12452_fix.json $C 2>&1 | grep -E "events_read|fired_Mass90|R9Unseeded|TrackIsoUnseeded
          |n_id22|events_with_unseeded|fraction|nTrigObj_mean|min|identical|Error"
242   python3 -c "import json; a=json.load(open(\"$O/wf12452_clean.json\")); b=json.load(open(\"$O/wf12452_fix.json\")); print(\"ids clean\", a[\"
          objects_per_id\"]); print(\"ids fix  \", b[\"objects_per_id\"]); print(\"id!=22 bits identical:\", a[\"bit_counts_per_id_without_22\"]==
          b[\"bit_counts_per_id_without_22\"]); print(\"photon bits clean\", a[\"photons_with_bit\"]); print(\"photon bits fix  \", b[\"
          photons_with_bit\"])"
243   python3 compare_dqm.py $(ls $W/run_clean/12452.0_*/DQM_V0001*.root) $(ls $W/run_fix/12452.0_*/DQM_V0001*.root) $O/wf12452_dqm_diff.json 2>&1 |
          head -60
244   echo "--- logs"
245   for A in clean fix; do L=$(ls $W/run_$A/12452.0_*/step3_*.log); echo "$A step3: Exception lines $(grep -c Exception $L), triggerObjectTable
          lines $(grep -c triggerObjectTable $L)"; grep triggerObjectTable $L | head -3; done
246   L=$(ls $W/run_clean/141.046_*/step3_*.log); echo "clean 141.046 step3: Exception lines $(grep -c Exception $L), triggerObjectTable lines $(
          grep -c triggerObjectTable $L)"' 2>&1 | grep -v '^\*' | cut -c1-300
247
248 # --- Rerun 12452.0 DQM comparison without time limit
249 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
250   source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
251   W=/eos/user/s/smahata/trigobj_check/relval_16_1_3; T=/eos/user/s/smahata/trigobj_check
252   cd $T && echo "start $(date)"
253   python3 compare_dqm.py $(ls $W/run_clean/12452.0_*/DQM_V0001*.root) $(ls $W/run_fix/12452.0_*/DQM_V0001*.root) nanoaod-trigobj-notes/results/
          relval_16_1_3/wf12452_dqm_diff.json > /dev/null 2>&1; echo "exit $? $(date)"
254   python3 -c "import json; d=json.load(open(\"nanoaod-trigobj-notes/results/relval_16_1_3/wf12452_dqm_diff.json\")); print(d[\"
          n_histograms_clean\"], d[\"n_histograms_fix\"], len(d[\"only_in_clean\"]), len(d[\"only_in_fix\"]), len(d[\"differing\"])); print(\"\n\".
          join(d[\"differing\"][:40]))"' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 4. Relval in CMSSW_16_1_3 (13/15)

slides/code/commands_as_run.sh, lines 256 to 277

```
256 # --- Convert and measure clean 141.046 NANO
257 timeout 1200 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=60 -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
258   source /cvmfs/cms.cern.ch/cmsset_default.sh; W=/eos/user/s/smahata/trigobj_check/relval_16_1_3; T=/eos/user/s/smahata/trigobj_check
259   (cd $W/clean/src && eval $(scramv1 runtime -sh) && cd $W/run_clean/141.046_RunEGamma2023D && \
260    cmsDriver.py flat --step NONE --data --conditions auto:run3_data_relval --era Run3_2023 \
261      --filein file:step3_inNANOEDMAOD.root --fileout file:step3_flatNANOAOD.root \
262      --eventcontent NANOAOD --datatier NANOAOD -n -1 > flat_nano.log 2>&1; echo "flat exit $?"; ls -l step3_flatNANOAOD.root)
263   source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh; cd $T
264   python3 measure_trigobj.py 16_1_3_clean wf141.046 $W/run_clean/141.046_RunEGamma2023D/step3_flatNANOAOD.root nanoaod-trigobj-notes/results/
          relval_16_1_3/wf141046_clean.json 2>&1 | grep -E "events_read|n_photon_bits|fired_Mass90|R9Unseeded|TrackIsoUnseeded|seeded_|n_id22|
          events_with_unseeded|nTrigObj_mean|Error"' 2>&1 | grep -v '^\*'
265
266 # --- Convert, measure, check logs and DQM for 141.046 fix
267 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
268   source /cvmfs/cms.cern.ch/cmsset_default.sh; W=/eos/user/s/smahata/trigobj_check/relval_16_1_3; T=/eos/user/s/smahata/trigobj_check; O=$T/
          nanoaod-trigobj-notes/results/relval_16_1_3
269   (cd $W/fix/src && eval $(scramv1 runtime -sh) && cd $W/run_fix/141.046_RunEGamma2023D && \
270    cmsDriver.py flat --step NONE --data --conditions auto:run3_data_relval --era Run3_2023 \
271      --filein file:step3_inNANOEDMAOD.root --fileout file:step3_flatNANOAOD.root \
272      --eventcontent NANOAOD --datatier NANOAOD -n -1 > flat_nano.log 2>&1; echo "flat exit $?")
273   L=$W/run_fix/141.046_RunEGamma2023D/step3_RunEGamma2023D.log; echo "fix 141.046 step3: Exception $(grep -c Exception $L), triggerObjectTable $
          (grep -c triggerObjectTable $L), events $(grep -c "Begin processing" $L)"
274   L=$W/run_clean/141.046_RunEGamma2023D/step3_RunEGamma2023D.log; echo "clean 141.046 step3 events $(grep -c "Begin processing" $L)"
275   source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh; cd $T
276   python3 measure_trigobj.py 16_1_3_fix wf141.046 $W/run_fix/141.046_RunEGamma2023D/step3_flatNANOAOD.root $O/wf141046_fix.json $W/run_clean
          /141.046_RunEGamma2023D/step3_flatNANOAOD.root 2>&1 | grep -E "events_read|fired_Mass90|n_id22|events_with_unseeded|nTrigObj_mean|
          events_in|identical|Error"
277   python3 compare_dqm.py $W/run_clean/141.046_RunEGamma2023D/DQM_V0001*.root $W/run_fix/141.046_RunEGamma2023D/DQM_V0001*.root $O/
          wf141046_dqm_diff.json > /dev/null 2>&1; echo "dqm exit $?"
```

Backup code: commands as run: 4. Relval in CMSSW_16_1_3 (14/15)

slides/code/commands_as_run.sh, lines 278 to 295

```
278   python3 -c "import json; d=json.load(open(\"$0/wf141046_dqm_diff.json\")); print(d[\"n_histograms_clean\"], d[\"n_histograms_fix\"], len(d[\"
          only_in_clean\"]), len(d[\"only_in_fix\"]), len(d[\"differing\"])); print(\"\n\".join(x for x in d[\"differing\"] if \"TimerService\"
          not in x)[:3000])"' 2>&1 | grep -v '^\*'
279
280 # --- Inspect trigger products in 141.046 MINIAOD and NANO
281 timeout 600 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=60 smahata@lxplus967.cern.ch '
282   source /cvmfs/cms.cern.ch/cmsset_default.sh; W=/eos/user/s/smahata/trigobj_check/relval_16_1_3
283   cd $W/clean/src && eval $(scram\v1 runtime -sh) && cd $W/run_clean/141.046_RunEGamma2023D
284   echo "== MINIAOD trigger products"; edmEventSize -v step3_inMINIAOD.root 2>/dev/null | grep -iE "trigger|TriggerResults" | head
285   echo "== EDM NANO TrigObj"; edmEventSize -v step3_inNANOEDMAOD.root 2>/dev/null | grep -iE "trig" | head
286   echo "== cfg"; grep -nE "unpackedPatTrigger|patTrigger\b|triggerResults|hltProcess|slimmedPatTrigger" step3_RAW2DIGI*.py 2>/dev/null | head;
          ls *.py' 2>&1 | grep -v '^\*' | cut -c1-200
287
288 # --- Check HLT trigger summary products in relval outputs
289 timeout 600 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=60 smahata@lxplus967.cern.ch '
290   source /cvmfs/cms.cern.ch/cmsset_default.sh; W=/eos/user/s/smahata/trigobj_check/relval_16_1_3
291   cd $W/clean/src && eval $(scram\v1 runtime -sh) && cd $W/run_clean/141.046_RunEGamma2023D
292   echo "== step2 (reHLT output) trigger summary products"; edmEventSize -v step2.root 2>/dev/null | grep -iE "TriggerEvent|hltTriggerSummary|
          TriggerResults" | head
293   echo "== step3 AOD"; edmEventSize -v step3.root 2>/dev/null | grep -iE "TriggerEvent|hltTriggerSummary" | head
294   cd $W/run_clean/12452.0_*; echo "== 12452 step2"; edmEventSize -v step2.root 2>/dev/null | grep -iE "TriggerEvent|hltTriggerSummary" | head;
          echo "== 12452 MINIAOD"; edmEventSize -v step3_inMINIAODSIM.root 2>/dev/null | grep -iE "slimmedPatTrigger" | head -3' 2>&1 | grep -v '
          ^\*' | cut -c1-200
```

Backup code: commands as run: 5. HiggsDNA photon-bit usage (15/15)

slides/code/commands_as_run.sh, lines 296 to 306

```
296 # ===== 5. HiggsDNA photon-bit usage
297
298 # --- Shallow-clone HiggsDNA repos and grep TrigObj usage
299 D=<scratch>/higgsdna_src; mkdir -p $D; cd $D; timeout 200 git clone -q --depth 1 https://gitlab.cern.ch/cms-analysis/general/HiggsDNA.git
        HiggsDNA 2>&1 | tail -2; timeout 200 git clone -q --depth 1 https://github.com/ZhenyuDongLL/HiggsDNA-bbggrun3.git bbgg 2>&1 | tail -2;
        git -C HiggsDNA log --oneline -1 | cat; git -C bbgg log --oneline -1 | cat; grep -rn --include=*.py --include=*.json -E "trigger_match|
        TrigObj|filterBits" HiggsDNA | grep -v "^HiggsDNA/tests/data" | cut -c1-220 | head -60
300
301 # --- Read HiggsDNA trigger-matching code and search bbgg fork
302 cd <scratch>/higgsdna_src; sed -n 230,262p HiggsDNA/higgs_dna/workflows/dystudies.py; echo ----; sed -n 300,325p HiggsDNA/higgs_dna/utils/
        misc_utils.py; echo ----; sed -n 630,645p HiggsDNA/higgs_dna/workflows/lowmass.py; echo ---- bbgg; grep -rn --include=*.py -E "
        trigger_match|TrigObj|filterBits" bbgg | cut -c1-200 | head -20; echo ---- other trig refs; grep -rln --include=*.py -iE "Diphoton30_22|
        hltEG22|Unseeded" HiggsDNA bbgg | head
303
304 # --- Show diphoton-trigger references in HiggsDNA systematics
305 cd <scratch>/higgsdna_src; grep -n -iE "Diphoton30_22|hltEG22|Unseeded" HiggsDNA/higgs_dna/systematics/event_weight_systematics.py HiggsDNA/
        higgs_dna/scripts/pull_files.py | cut -c1-200 | head -12
```